UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería

# AVANCE DE PROYECTO FINAL 2

## AYNI BANK · Banca 100 % digital

**Curso:**

Integrador II: Software

**Docente:**

Dr. Carlos R. P. Tovar

**Grupo:**

01 · sección 35362

**Integrantes:**


1. Fabian García Champi — Cod. U21201692
2. Joaquin Alfonso Loa Denegri — Cod. U22234069
3. Kiara Mishell Santti Saavedra — Cod. U22201636
4. Gerardo Raul Socualaya Mandamiento — Cod. U22229068
5. Frank Grheg Sotomayor Suasnabar — Cod. U18208457
6. Eduardo Vargas Zumaeta — Cod. U22234544


Lima, lunes 05 de octubre del 2026

# Índice


INTRODUCCIÓN ..... 4

Cómo leer este documento ..... 4

Enlaces del proyecto ..... 4

1. LEVANTAMIENTO DE OBSERVACIONES DEL APF1 ..... 6

1.1 Cronograma: línea base contra ejecución real ..... 6

1.2 Desglose de tareas y tablero ..... 8

1.3 Evolución del sistema entre el APF1 y el APF2 ..... 10

2. INTEGRACIÓN CON BASE DE DATOS ..... 11

2.1 Informe de administración y replicación ..... 11

2.1.1 Motor y organización ..... 11

2.1.2 Estrategia de replicación ..... 11

2.2 Diseño físico de la base de datos ..... 14

2.2.1 Tablas nuevas ..... 16

2.2.2 Invariantes del modelo ..... 16

2.2.3 Partida doble también con otros bancos ..... 16

2.2.4 Auditoría en core ..... 16

2.3 Implementación del patrón de acceso a datos ..... 18

2.3.1 Diagrama de secuencia: inicio de sesión ..... 20

2.3.2 Justificación técnica ..... 22

2.4 Consistencia entre el modelo y el script SQL ..... 22

Fuentes consultadas ..... 25

3. MEDIDAS DE SEGURIDAD DE LA APLICACIÓN ..... 26

3.1 Módulo de autenticación y autorización ..... 26

3.2 Informe técnico de seguridad y cifrado de datos ..... 29

3.2.1 Confidencialidad: cifrado ..... 29

3.2.2 Integridad y trazabilidad ..... 29

3.2.3 Disponibilidad ..... 29

3.3 Prueba de seguridad web ..... 31

3.4 Catálogo de controles de seguridad ..... 35

Fuentes consultadas ..... 35

4. DESPLIEGUE DE LA APLICACIÓN EN LA NUBE ..... 36

4.1 Plan de pruebas del sistema ..... 36

4.2 Manual de despliegue ..... 40

4.3 Evidencia de pruebas de despliegue ..... 42

4.3.1 Lo que falló y cómo respondió el sistema ..... 44

4.3.2 De la 1.1.1 a la 1.1.6: corregir en producción con el mismo pipeline ..... 47

4.3.3 Infraestructura creada en AWS ..... 50
4.4 Monitoreo y administración de la base de datos en producción ..... 54
4.4.1 La base de datos en Amazon RDS ..... 54
4.4.2 Alarmas ..... 60
4.4.3 Salud de la aplicación en Grafana ..... 62
4.4.4 Tareas de administración ..... 64
4.4.5 Los datos de producción en la base ..... 64
Fuentes consultadas ..... 66
5. VALIDACIÓN DEL SISTEMA ..... 67
5.1 En producción: una cuenta abierta por el recorrido real ..... 67
5.2 En el entorno local: el recorrido completo de las operaciones ..... 71
5.2.1 Punto de partida ..... 71
5.2.2 Depósito ..... 73
5.2.3 Transferencia entre cuentas Ayni ..... 75
5.2.4 Rechazo por límite ..... 79
5.2.5 Resultado final ..... 81
6. TECNOLOGÍAS UTILIZADAS EN CADA PUNTO ..... 83
7. CONCLUSIONES ..... 84

# INTRODUCCIÓN

Ayni Bank es una banca 100 % digital para personas naturales en el Perú: cuenta de ahorros sin comisión de mantenimiento, apertura en minutos con verificación de identidad por fotografía del DNI y selfie, y operaciones desde el celular. En el primer avance (APF1) se presentó el análisis del negocio, la planificación, las herramientas, los prototipos y la arquitectura. Este segundo avance (APF2) responde a una pregunta distinta y más exigente: **¿el sistema funciona de verdad, en la nube, con su base de datos y protegido?**

La respuesta corta es sí, y se puede comprobar. El sistema está publicado en Amazon Web Services en **https://ayni.produccion.ejemplo**; cada cambio que entra a la rama `main` se construye, se prueba, se escanea, se despliega y se verifica solo, y si la verificación falla el pipeline devuelve producción a la versión anterior. Eso ocurrió de verdad durante la publicación de la versión 1.1.0 y está documentado como evidencia en el capítulo 4. En producción ya hay una cuenta abierta por el recorrido real de un cliente (onboarding con DNI real, aprobación y primer depósito); las transferencias, que necesitan dos cuentas, se validaron de punta a punta en el entorno local con la misma base de código. Las capturas de ambos están en el capítulo 5.

## Cómo leer este documento

El informe sigue el orden exacto de la guía del APF2. La tabla indica dónde está cada artefacto de la rúbrica para que pueda verificarse sin recorrer el documento entero.

**Tabla 1** · Matriz de trazabilidad entre la rúbrica del APF2 y el informe

| Criterio de la rúbrica | Artefacto exigido | Dónde está |
|---|---|---|
| Levantamiento de observaciones (4 pts) | Cuadro de observaciones y evidencia | Capítulo 1 |
| Integración con base de datos (4 pts) | Informe de administración y replicación | 2.1 |
| | Diseño físico de la base de datos | 2.2 |
| | Patrón de acceso a datos y diagrama de secuencia | 2.3 |
| | Consistencia entre el modelo y el script SQL | 2.4 |
| Medidas de seguridad (4 pts) | Módulo de autenticación y autorización | 3.1 |
| | Informe técnico de seguridad y cifrado | 3.2 |
| | Prueba de seguridad web | 3.3 |
| | Catálogo de controles de seguridad | 3.4 |
| Despliegue de la aplicación (4 pts) | Plan de pruebas del sistema | 4.1 |
| | Manual de despliegue | 4.2 |
| | Evidencia de pruebas de despliegue | 4.3 |
| | Monitoreo y administración de la base de datos | 4.4 |
| Validación en la nube (2 pts) | Funcionamiento con base de datos y seguridad | Capítulo 5 |
| Documento de tecnologías | Tecnología usada en cada punto | Capítulo 6 |

*Nota.* Elaboración propia con base en la guía del Avance de Proyecto Final 2 del curso.

## Enlaces del proyecto

Todo lo que se describe en este informe se puede comprobar directamente. El sistema está publicado en internet, el código es público y la gestión del proyecto es visible en Jira:

- **Producción:** la aplicación funcionando en AWS, la misma que se muestra en las figuras.
- **Repositorio:** el código, las migraciones de la base, los ADR y el pipeline de GitHub Actions.
- **Jira:** el backlog, los sprints y el estado de cada historia y tarea.
- **Grafana:** el tablero de salud de la aplicación en producción.

**Tabla 2** · Enlaces de acceso a los recursos del proyecto

| Recurso | Enlace |
|---|---|
| Sistema en producción (AWS) | https://ayni.produccion.ejemplo |
| Repositorio de código | https://github.com/LOAD-13/ayni-bank |
| Pipeline CI/CD | https://github.com/LOAD-13/ayni-bank/actions |
| Releases | https://github.com/LOAD-13/ayni-bank/releases |
| Tablero ágil (Jira) | https://jloadenegri.atlassian.net/jira/software/projects/AYNI |
| Monitoreo (Grafana) | https://ayni.produccion.ejemplo/grafana |
| Calidad (SonarCloud) | https://sonarcloud.io/project/overview?id=LOAD-13_ayni-bank |

*Nota.* Elaboración propia. Todos los enlaces son públicos y permiten verificar directamente lo descrito en el informe.

# 1. LEVANTAMIENTO DE OBSERVACIONES DEL APF1

La guía pide empezar por aquí, y tiene sentido: un avance que no corrige lo señalado en el anterior repite los mismos errores con más código. Recogimos las observaciones de la sustentación del APF1 y de la revisión de la base de datos de la semana 6, las convertimos en tareas de Jira y cada una tiene su evidencia verificable.

**Tabla 3** · Observaciones del APF1 y cómo se levantaron

| N.º | Observación | Cómo se levantó | Estado |
|---|---|---|---|
| 1 | Actualizar el cronograma (Gantt) contra la línea base. | Gantt regenerado al 3 de octubre: sprints 0 a 2 cerrados, Sprint 3 adelantado. Ver 1.1. | Levantada |
| 2 | Mantener al día el desglose de tareas en Jira. | Historias y tareas desplegadas pasadas a Listo y Sprint 2 cerrado. Ver 1.2. | Levantada |
| 3 | Reflejar el cambio de Product Owner (el docente) en el Acta. | Acta de Constitución v3: el docente es el Product Owner y Joaquín pasa a Scrum Master. | Levantada |
| 4 | Base de datos con relaciones y restricciones explícitas. | Diccionario regenerado desde el catálogo real de RDS: 25 tablas, 18 relaciones, 58 restricciones. Ver 2.2. | Levantada |
| 5 | ¿Una transferencia desde otro banco crea un solo asiento? | No: siempre dos. La contrapartida es la cuenta de compensación interbancaria (CCE). Ver 2.2.3. | Levantada |
| 6 | Falta una tabla de auditoría en core, como la de identity. | Nueva tabla core.evento_auditoria con operaciones y rechazos. Ver 2.2.4. | Levantada |
| 7 | La seguridad debe presentarse en tablas, no en texto corrido. | Catálogo de controles y matriz de madurez en tablas. Ver 3.2 y 3.4. | Levantada |
| 8 | Demostrar desde localhost anula los puntos de despliegue. | Sistema público en AWS con despliegue continuo demostrable en vivo. Ver capítulo 4. | Levantada |

*Nota.* Elaboración propia a partir de la retroalimentación de la sustentación del APF1 y de la revisión de la base de datos de la semana 6.

## 1.1 Cronograma: línea base contra ejecución real

La línea base se fijó el 23 de agosto y no se ha movido: es lo que permite medir la desviación. La barra discontinua es lo planificado y la maciza lo que realmente ocurrió. El Sprint 2 se alargó seis días porque estaba sobrecomprometido —lo advertimos en el APF1—, pero cerró con el onboarding completo (DNI por OCR y selfie). A cambio, el Sprint 3 se adelantó: las transferencias (HU-07), el saldo y los movimientos (HU-08) y la producción en AWS estuvieron listos antes de esta entrega, cuando el plan los situaba en el hito «Producción v1» del 4 de octubre.

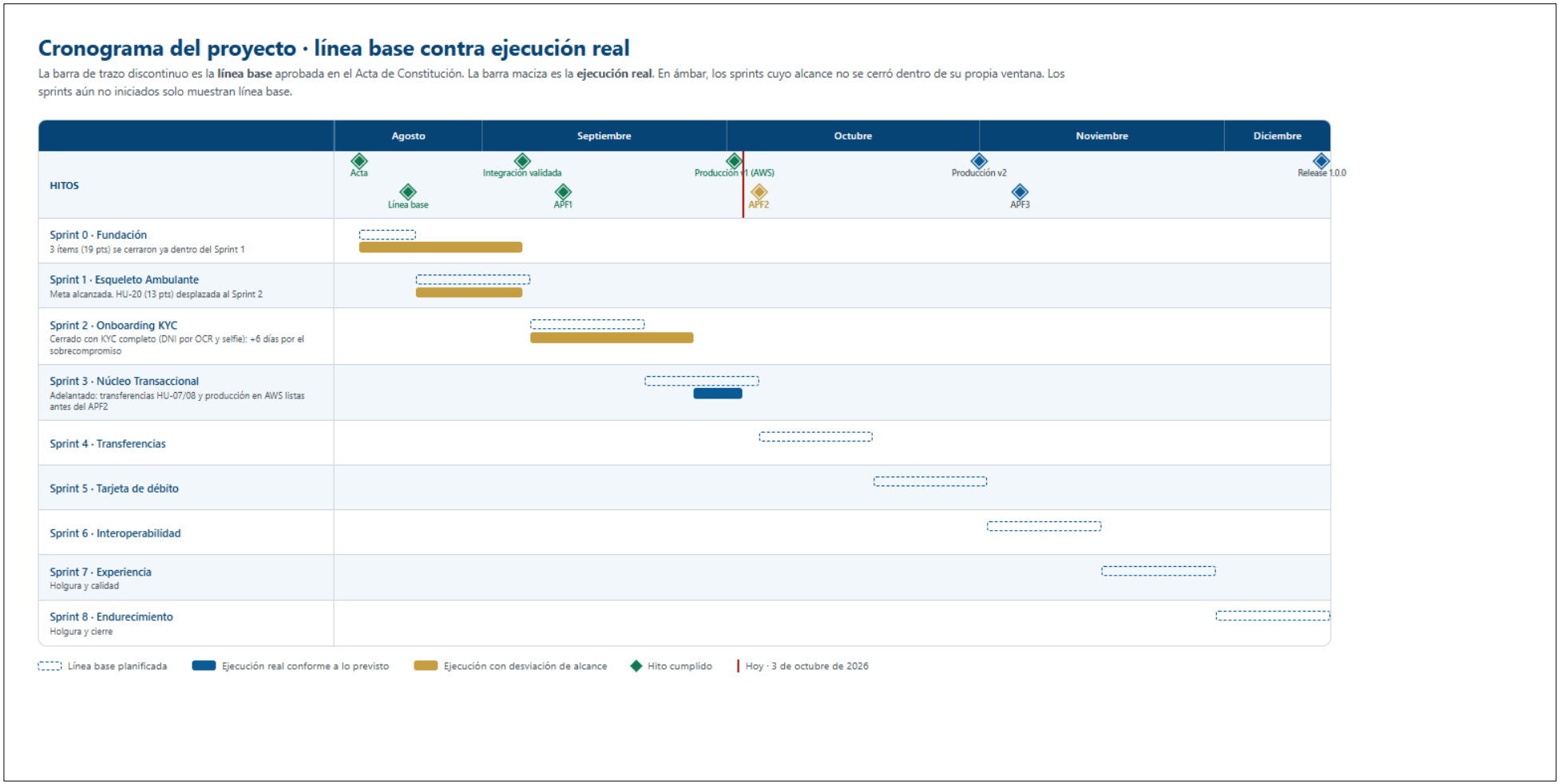



Figura 1 · Gantt híbrido PMBOK + sprints, al 3 de octubre de 2026.

## 1.2 Desglose de tareas y tablero

Cada cambio entró por un Pull Request revisado y enlazado a su tarea de Jira. En este avance se crearon las tareas AYNI-157 (pruebas de seguridad automatizadas), AYNI-158 (auditoría y compensación interbancaria) y AYNI-159 (Prometheus y Grafana), y se cerraron HU-07, HU-08, HU-20 y T-12.

El Sprint 2 se cerró con 55 puntos completados. La HU-22 (segundo factor dentro del registro) quedó en curso: el paso «Seguridad» antes del DNI y el bloqueo de la captura sin segundo factor no están hechos, así que pasó al Sprint 3 en lugar de darse por terminada.

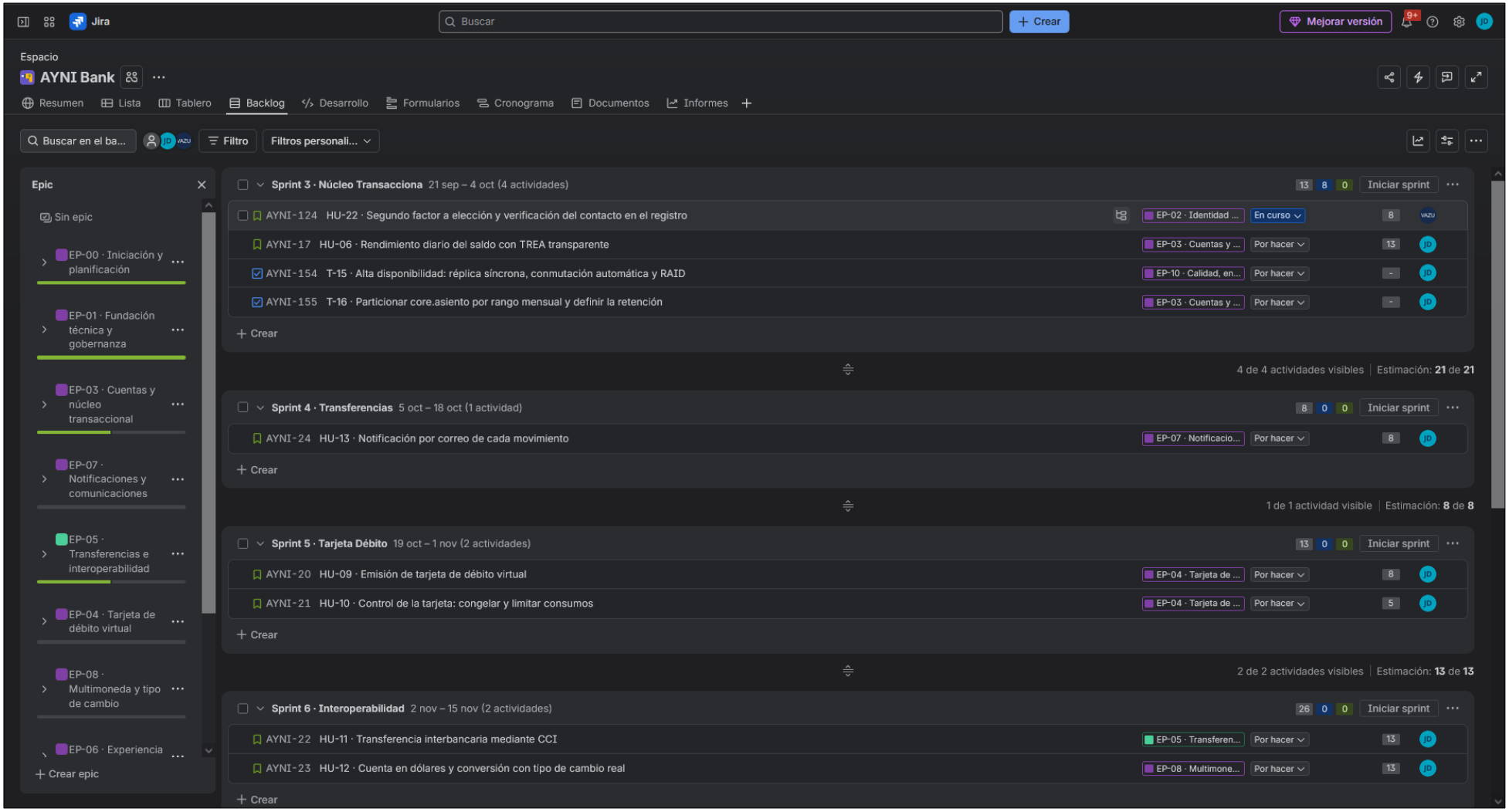



Figura 2 · Backlog tras cerrar el Sprint 2: la HU-22 pasa al Sprint 3, seguido de los sprints planificados.

## 1.3 Evolución del sistema entre el APF1 y el APF2

Entre el primer y el segundo avance el proyecto pasó de estar diseñado a estar funcionando. La tabla compara los dos momentos en los aspectos que evalúa este avance. Los cambios más importantes son tres:

- **De un despliegue diseñado a uno real:** el plan original (Raspberry Pi y Oracle Cloud) se reemplazó por AWS, con despliegue continuo y reversión automática.
- **Una base de datos más completa:** de 21 a 25 tablas, con auditoría en los dos schemas y la cuenta de compensación interbancaria.
- **Seguridad centralizada:** el token se valida una sola vez en el gateway, y la seguridad se prueba con herramientas de Kali contra producción.

**Tabla 4** · Evolución del sistema entre el APF1 y el APF2

| Aspecto | APF1 (semana 5) | APF2 (semana 9) |
|---|---|---|
| Despliegue | Diseñado para Raspberry Pi y Oracle Cloud | AWS en funcionamiento con despliegue continuo y rollback automático |
| Base de datos | 21 tablas en PostgreSQL local | 25 tablas en Amazon RDS, con auditoría en los dos schemas |
| Autenticación | JWT validado en cada servicio | JWT centralizado en el gateway y segundo factor a elección |
| Correo | Solo registros en el log | Correos reales con Amazon SES y plantilla de marca |
| Onboarding | Selfie; DNI pendiente | OCR del DNI por lado y confirmación del titular |
| Transacciones | Solo apertura de cuenta | Panel, depósito, transferencia idempotente y comprobante |
| Monitoreo | Plan de métricas | Prometheus y Grafana más alarmas de CloudWatch |
| Pruebas de seguridad | Diseñadas | OWASP ZAP, Nmap, Nikto y testssl contra producción |

*Nota.* Elaboración propia. El APF1 corresponde a la semana 5 y el APF2 a la semana 9 del curso.

# 2. INTEGRACIÓN CON BASE DE DATOS

El criterio pide que la base de datos sea coherente con los requerimientos y que las consultas, las operaciones y la replicación estén bien configuradas. Lo que más nos importa de esa frase es «coherente»: en una banca, la base no es un almacén de datos, es la garantía de que el dinero cuadra.

## 2.1 Informe de administración y replicación

Esta sección describe cómo se administra la base de datos en producción y cómo se protege frente a fallos. Se organiza en dos partes:

- **Motor y organización:** qué motor usamos, por qué, y cómo se separan los datos de cada servicio.
- **Replicación:** qué protección tiene hoy la base y qué estrategia proponemos para una operación bancaria real.

### 2.1.1 Motor y organización

Usamos **PostgreSQL 17.11 administrado por Amazon RDS** (instancia db.t4g.micro, cifrada con KMS). Elegimos PostgreSQL por tres razones que se pueden medir: transacciones ACID con bloqueo por fila (`SELECT … FOR UPDATE`), tipos exactos para dinero (`NUMERIC`) y restricciones `CHECK` que el motor hace cumplir aunque la aplicación falle. RDS añade lo que una base en una máquina propia no da: copias automáticas, recuperación a un punto en el tiempo y parches sin intervención.

Una sola base `ayni` contiene un **schema por contexto de negocio**: `identity` (usuarios, onboarding, sesiones), `core` (cuentas, asientos, operaciones) y `notification`. Cada microservicio migra y lee solo su schema; entre schemas no hay claves foráneas, sino proyecciones locales por identificador. Así un servicio puede cambiar sus tablas sin romper a otro, y la frontera de los microservicios es también la frontera de los datos.

**Tabla 5** · Prácticas de administración de la base de datos

| Práctica de administración | Cómo se aplica en Ayni Bank |
|---|---|
| Usuarios y privilegios | La aplicación usa `ayni_app`, sin privilegios de superusuario. La cuenta maestra la genera y custodia Secrets Manager; nadie la conoce. |
| Cambios de esquema | Solo por migraciones Flyway versionadas en git y revisadas en un PR (identity V1–V8, core V1–V4). Se aplican solas al desplegar. |
| Respaldo | Copias automáticas diarias de RDS con recuperación a un punto en el tiempo (PITR) y snapshot final al eliminar la instancia. |
| Acceso de red | RDS no tiene IP pública: solo acepta conexiones desde el grupo de seguridad de la EC2, y siempre con TLS (`sslmode=require`). |
| Monitoreo | Métricas de CloudWatch (CPU, conexiones, almacenamiento, memoria) con alarmas por correo; pool de conexiones visible en Grafana. |
| Datos sensibles | Contraseñas con Argon2id; número de DNI y secreto TOTP cifrados con AES-256-GCM; solo los últimos 4 dígitos en claro. |

*Nota.* Elaboración propia con base en la configuración de Amazon RDS en producción.

### 2.1.2 Estrategia de replicación

Hoy la base corre en una sola zona de disponibilidad (Single-AZ) porque el plan gratuito de AWS no cubre Multi-AZ; lo compensamos con copias diarias y PITR. Para una operación bancaria real la estrategia propuesta tiene tres niveles, y cada uno resuelve un riesgo distinto:

**Tabla 6** · Estrategia de replicación propuesta por niveles

| Nivel | Tipo | Para qué | Justificación técnica |
|---|---|---|---|
| 1 | Multi-AZ: réplica síncrona en otra zona | Alta disponibilidad | La escritura se confirma en las dos zonas antes de responder; si cae la primaria, RDS conmuta en 60–120 s sin perder transacciones (RPO = 0). Un banco no puede perder un asiento confirmado. |

| Nivel | Tipo | Para qué | Justificación técnica |
|---|---|---|---|
| 2 | Réplica de lectura asíncrona | Escalar las lecturas | El panel y el historial son lecturas: se desvían a la réplica y la primaria solo escribe. Unos segundos de retraso son aceptables en una consulta, nunca en un saldo para transferir. |
| 3 | Copia de snapshots a otra región | Recuperación ante desastre | Si cae la región us-east-1 completa, se restaura en otra a partir de los snapshots copiados. |

*Nota.* Elaboración propia. Hoy la base opera en una sola zona (Single-AZ); los niveles se proponen para una operación bancaria real.

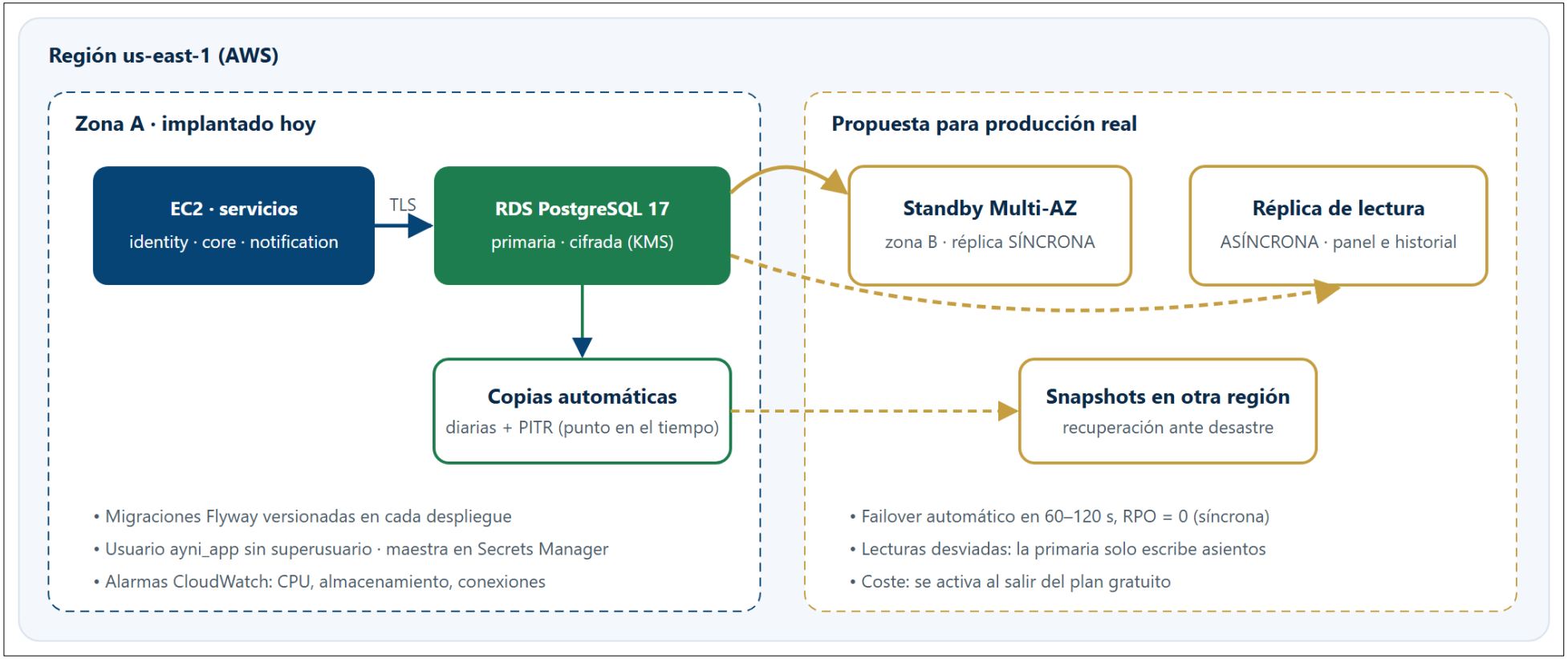



Figura 3 · Arquitectura de datos: lo implantado y la replicación propuesta.

## 2.2 Diseño físico de la base de datos

El diseño físico no se dibuja a mano: el diccionario de datos y el diagrama se generan **leyendo el catálogo real** (`information_schema`) de la base de producción en RDS. Si el diagrama dice que existe una restricción, es porque la base la tiene. Frente al APF1, el modelo crece en cuatro tablas y 34 columnas; las nuevas están marcadas como «NUEVO APF2».

**Tabla 7** · Comparación del modelo de datos entre el APF1 y el APF2

| | APF1 | APF2 | Diferencia |
|---|---|---|---|
| Tablas | 21 | 25 | +4 |
| Columnas | 144 | 178 | +34 |
| Relaciones | 14 | 18 | +4 |
| Restricciones | — | 58 | |
| Índices | — | 55 | |

*Nota.* Elaboración propia a partir del catálogo information_schema de la base de producción en Amazon RDS.

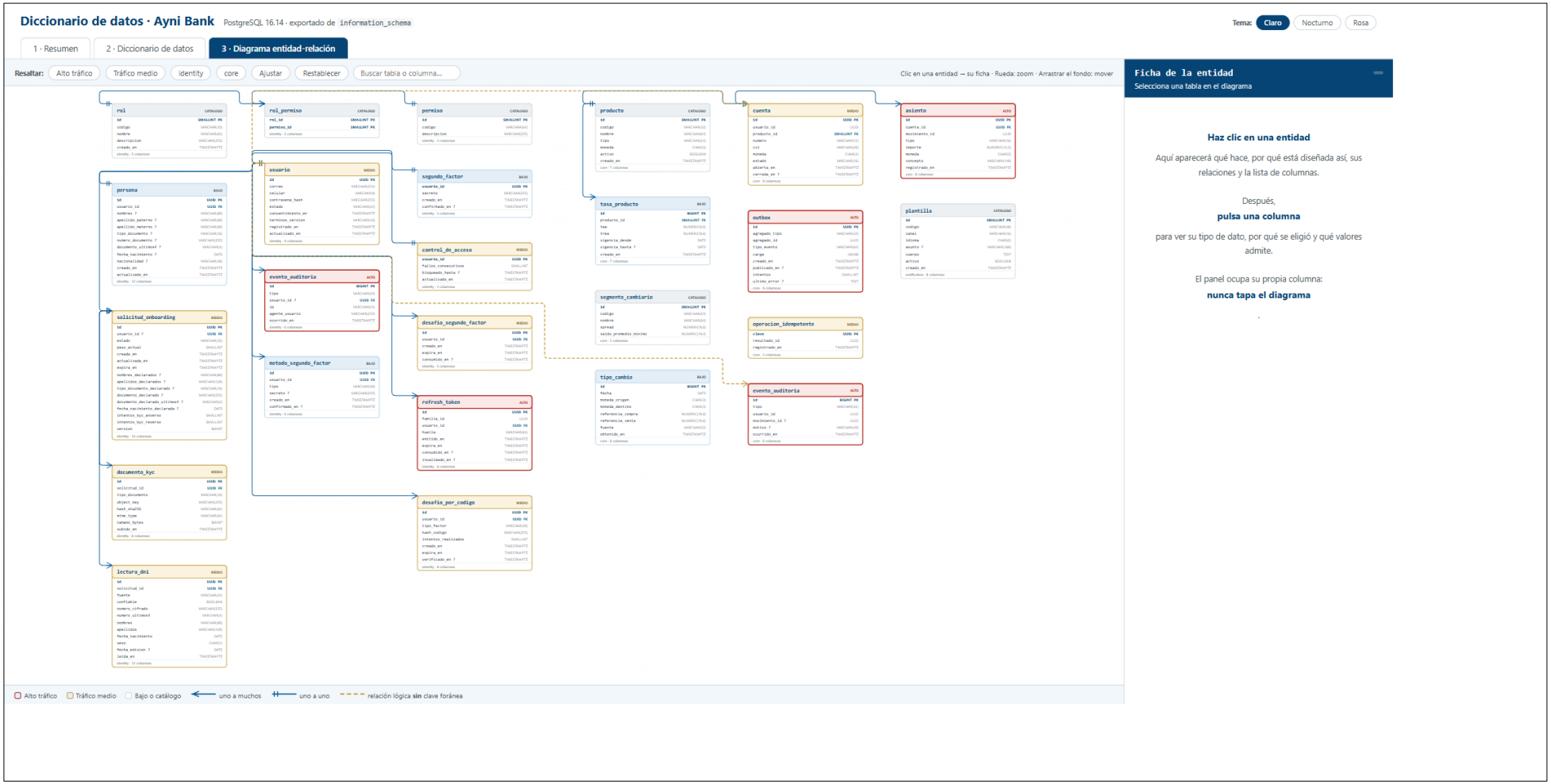



Figura 4 · Diagrama entidad-relación de producción (APF2).

### 2.2.1 Tablas nuevas

Respecto al diccionario de datos de la semana 6 se añadieron cuatro tablas. Cada una nace de una historia o de una observación concreta, y todas declaran sus restricciones en la propia base, de modo que un error en la aplicación no puede dejar datos incoherentes:

**Tabla 8** · Tablas nuevas del modelo de datos

| Tabla | Para qué | Restricciones clave |
|---|---|---|
| identity.metodo_segundo_factor | Qué segundos factores tiene cada usuario (app, correo, SMS) | UNIQUE (usuario_id, tipo); secreto cifrado |
| identity.desafio_por_codigo | Códigos de un solo uso por correo, con caducidad | Solo el hash del código; CHECK de 3 intentos |
| identity.lectura_dni | Datos que leyó el OCR y los que confirmó el titular | Número de DNI cifrado; CHECK de fuente y sexo |
| core.evento_auditoria | Pista de auditoría de operaciones, incluidos los rechazos | CHECK de coherencia entre tipo, motivo y movimiento |

*Nota.* Elaboración propia a partir del diccionario de datos generado desde la base de producción.

### 2.2.2 Invariantes del modelo

- **El saldo no se guarda**: es la suma de los asientos de la cuenta. No hay columna «saldo» que alguien pueda editar.
- **Todo movimiento son dos asientos** —un cargo y un abono— que suman cero.
- **Toda operación es idempotente**: la clave de idempotencia es clave primaria; un duplicado lo rechaza el motor.
- **Todo importe es NUMERIC** y en Java `BigDecimal` con redondeo HALF_EVEN; nunca `float` ni `double`.

### 2.2.3 Partida doble también con otros bancos

En la revisión de la base de datos el docente preguntó qué ocurre cuando una transferencia llega desde otro banco. En la sustentación respondimos que se creaba un solo asiento, y **esa respuesta era incorrecta**: con un solo asiento el libro deja de sumar cero y ya no se puede conciliar con la Cámara de Compensación Electrónica (CCE). Lo corregimos así (ADR-0029): cuando la contraparte está fuera de Ayni Bank, la representa una **cuenta técnica de compensación interbancaria**, sembrada por la migración V4.

**Tabla 9** · Asientos de partida doble por tipo de operación

| Operación | CARGO | ABONO |
|---|---|---|
| Transferencia entre cuentas Ayni | Cuenta del ordenante | Cuenta del beneficiario |
| Depósito simulado | Cuenta técnica de fondeo (V3) | Cuenta del cliente |
| Transferencia que llega de otro banco | Cuenta de compensación CCE (V4) | Cuenta del cliente |
| Transferencia hacia otro banco | Cuenta del cliente | Cuenta de compensación CCE (V4) |

*Nota.* Elaboración propia. Cada operación registra un cargo y un abono por el mismo importe.

El saldo de la cuenta de compensación es, en cada momento, la posición neta frente a la CCE: al liquidar cada ciclo vuelve a cero, y si no vuelve hay una diferencia que conciliar.

### 2.2.4 Auditoría en core

La segunda observación fue que core no tenía pista de auditoría, como sí la tenía identity. La nueva tabla `core.evento_auditoria` registra cada transferencia, depósito, petición repetida y rechazo con su motivo. Los rechazos importan especialmente: no dejan asientos, así que sin esta tabla no habría forma de saber cuántas veces alguien intentó transferir sin saldo. Se escriben en una transacción propia (`REQUIRES_NEW`) para que el rollback de la operación no borre su rastro. No guarda importes ni números de cuenta: el importe ya está en los asientos.

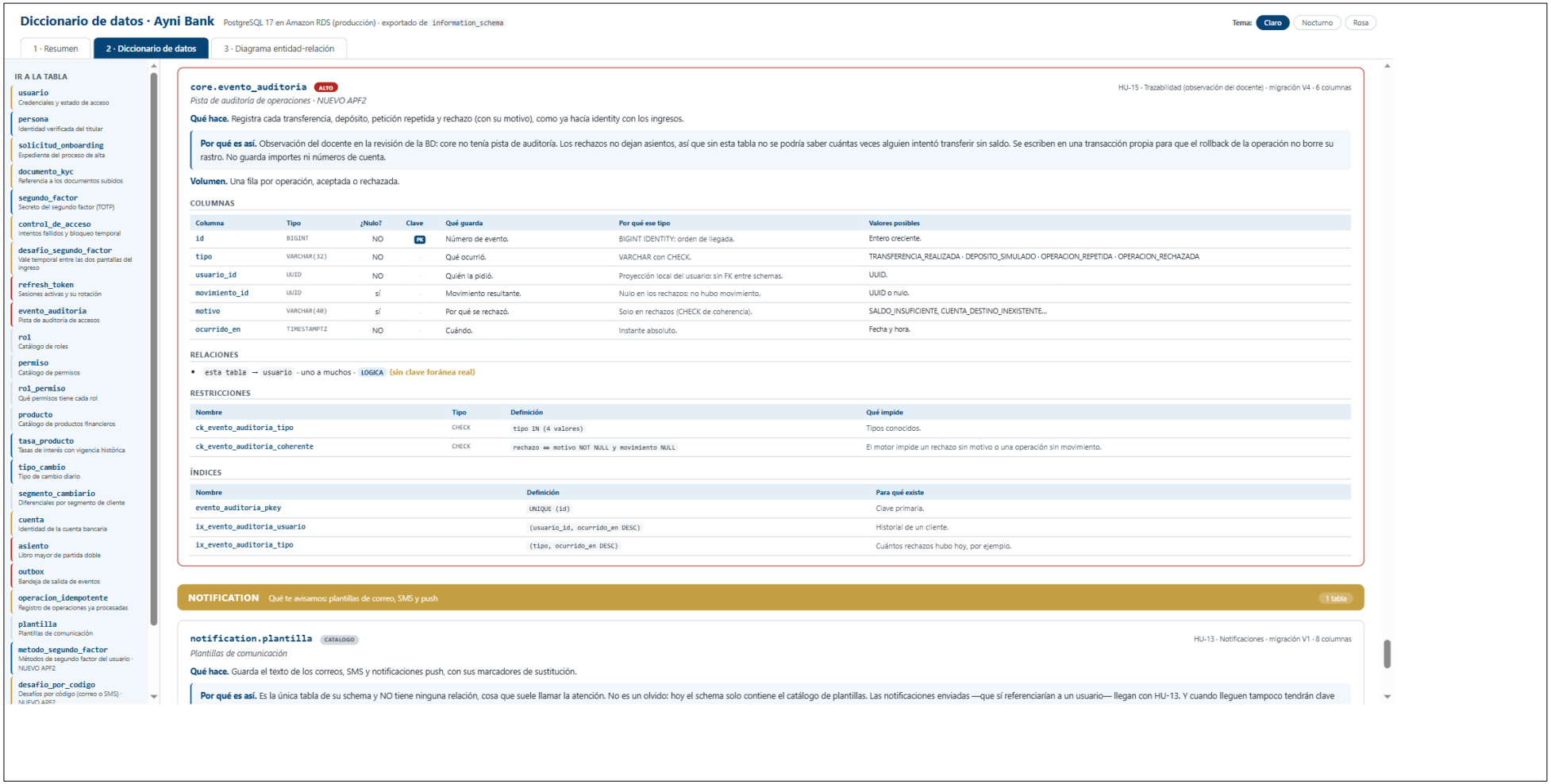



Figura 5 · Ficha de la nueva tabla de auditoría en el diccionario de datos.

## 2.3 Implementación del patrón de acceso a datos

Cada servicio aplica el patrón **Repository** dentro de una arquitectura hexagonal. El dominio declara lo que necesita de la persistencia como una interfaz —un puerto de salida— sin saber que existe una base de datos. Un adaptador de infraestructura implementa ese puerto con **Spring Data JPA sobre Hibernate**, y traduce entre las filas (entidades JPA) y los objetos del dominio. La regla «el dominio no importa ningún framework» no es un acuerdo: ArchUnit la comprueba en cada build y lo rompe si alguien la viola.

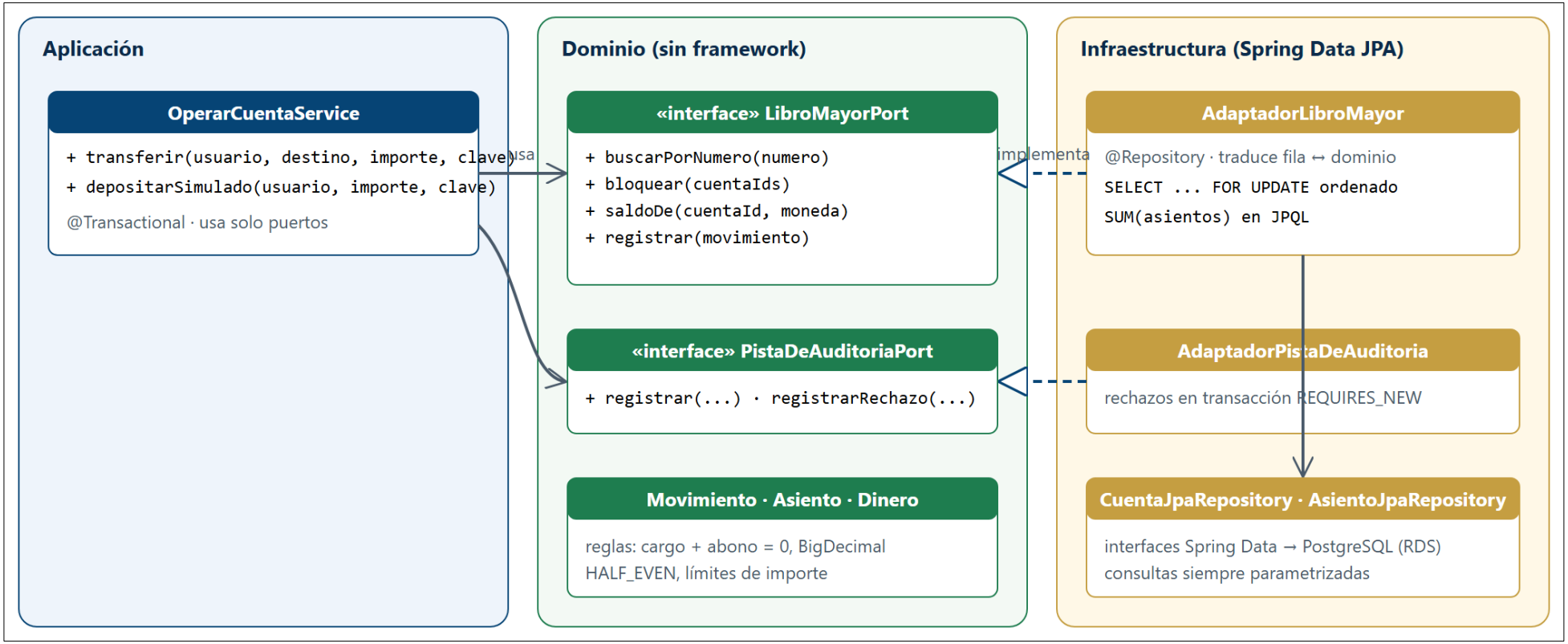



Figura 6 · Diagrama de clases del patrón Repository en core-banking.

### 2.3.1 Diagrama de secuencia: inicio de sesión

El diagrama recorre una petición real de punta a punta: la vista (Next.js) llama al gateway, que la enruta al controlador; el caso de uso pide el usuario al puerto; el adaptador lo busca con Spring Data JPA, Hibernate genera la consulta parametrizada contra PostgreSQL y la fila vuelve convertida en objeto del dominio. El caso de uso verifica la contraseña con Argon2id, registra el desafío del segundo factor y el evento de auditoría, y solo al validar el código TOTP emite el token.

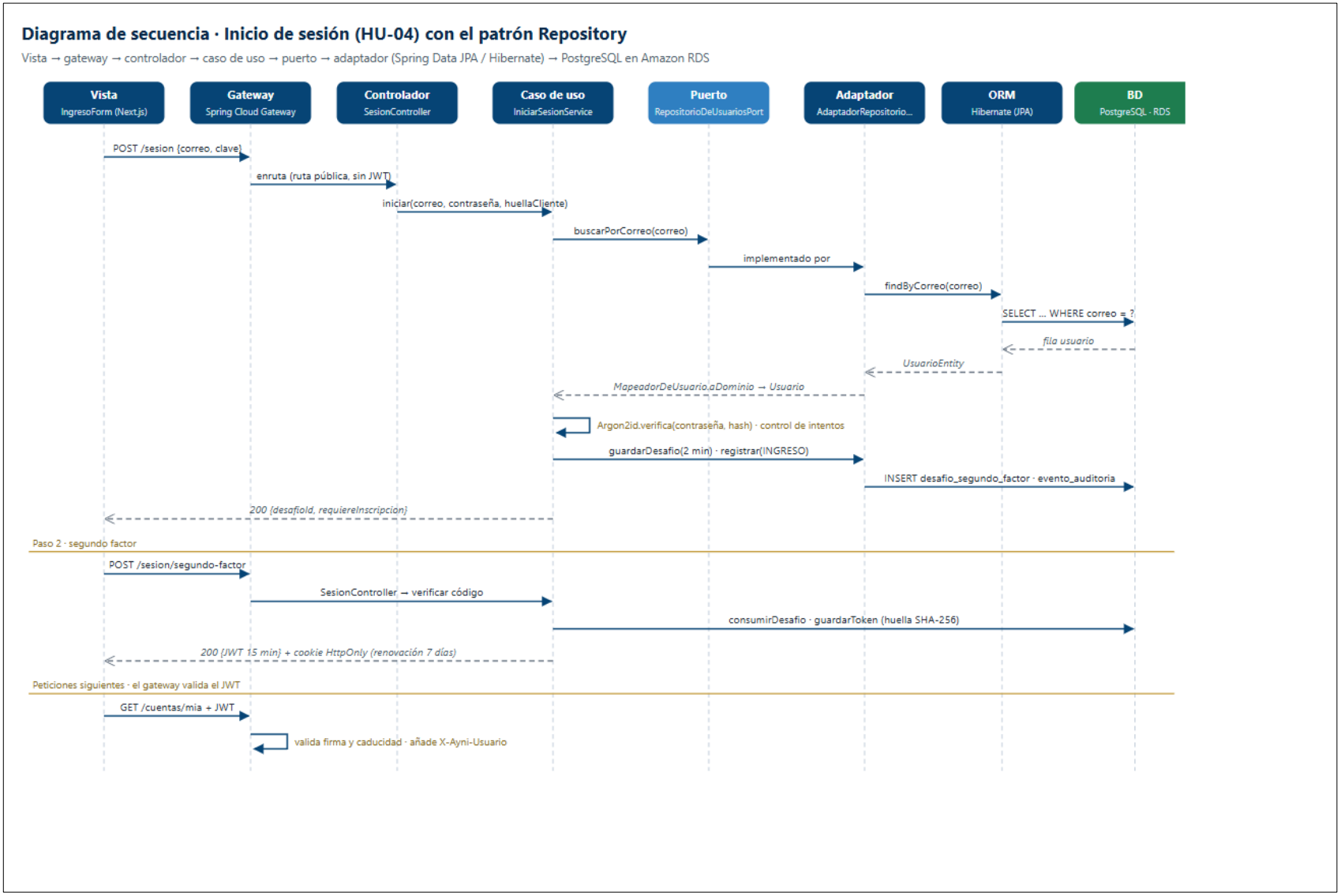



Figura 7 · Diagrama de secuencia del inicio de sesión: vista, controlador, caso de uso, repositorio, Hibernate y base de datos.

```
core-banking · puerto del dominio y su adaptador JPA
public interface LibroMayorPort {                // dominio: sin JPA ni SQL
    Optional<Cuenta> buscarPorNumero(NumeroDeCuenta numero);
    void bloquear(List<UUID> cuentaIds);          // SELECT ... FOR UPDATE ordenado
    Dinero saldoDe(UUID cuentaId, Moneda moneda);  // suma de asientos
    void registrar(Movimiento movimiento);         // los dos asientos juntos
}

@Repository
public class AdaptadorLibroMayor implements LibroMayorPort {
    private final CuentaJpaRepository cuentas;     // Spring Data JPA
    private final AsientoJpaRepository asientos;
    ...
}
```

### 2.3.2 Justificación técnica

Cada decisión del acceso a datos responde a un riesgo concreto del negocio bancario: que una consulta abra la puerta a una inyección SQL, que dos operaciones simultáneas descuadren un saldo o que un reintento mueva el dinero dos veces. La tabla explica qué decidimos y por qué:

**Tabla 10** · Decisiones técnicas del acceso a datos

| Decisión | Por qué |
|---|---|
| Repository y no DAO genérico | El repositorio habla el idioma del negocio (`saldoDe`, `bloquear`); un DAO habla de filas. El caso de uso no cambia si mañana el saldo sale de otra fuente. |
| ORM (JPA/Hibernate) para el CRUD | Consultas parametrizadas por defecto, sin inyección SQL, y mapeo explícito fila ↔ dominio. |
| JPQL para el saldo | La suma de asientos se calcula en la base; no se traen miles de filas a memoria. |
| Bloqueo pesimista ordenado por id | Dos transferencias cruzadas no producen interbloqueo ni saldo negativo. |
| Clave de idempotencia como PK | Si la red reintenta, el motor rechaza el duplicado: el dinero no se mueve dos veces. |

*Nota.* Elaboración propia a partir de la arquitectura del servicio core-banking.

## 2.4 Consistencia entre el modelo y el script SQL

La consistencia está garantizada por construcción y no por revisión manual: el DDL son las migraciones Flyway del repositorio, se aplican solas en cada despliegue, el diagrama se genera leyendo lo que esas migraciones crearon en RDS, y al arrancar Hibernate valida que cada entidad coincide con su tabla (`ddl-auto: validate`): una entidad desalineada impide que el servicio arranque y el pipeline revierte.

```
core-banking · V4__auditoria_y_cuenta_de_compensacion.sql
CREATE TABLE evento_auditoria (
    id            BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tipo          VARCHAR(32) NOT NULL,
    usuario_id    UUID        NOT NULL,
    movimiento_id UUID,
    motivo        VARCHAR(40),
    ocurrido_en   TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT ck_evento_auditoria_coherente
        CHECK ((tipo = 'OPERACION_RECHAZADA' AND motivo IS NOT NULL AND movimiento_id IS NULL)
            OR (tipo <> 'OPERACION_RECHAZADA' AND motivo IS NULL AND movimiento_id IS NOT NULL))
);
```

**Tabla 11** · Migraciones Flyway aplicadas en producción por servicio

| Servicio | Migraciones aplicadas en producción |
|---|---|
| identity | V1 roles y permisos · V2 usuario y onboarding · V3 datos declarados · V4 sesión y segundo factor · V5 documentos KYC · V6 límite de intentos · V7 segundo factor a elección · V8 lectura del DNI |

| Servicio | Migraciones aplicadas en producción |
|---|---|
| core-banking | V1 productos y tipo de cambio · V2 cuenta, asientos y outbox · V3 cuenta de fondeo · V4 auditoría y compensación CCE |

*Nota.* Elaboración propia a partir del historial de migraciones Flyway de cada servicio.

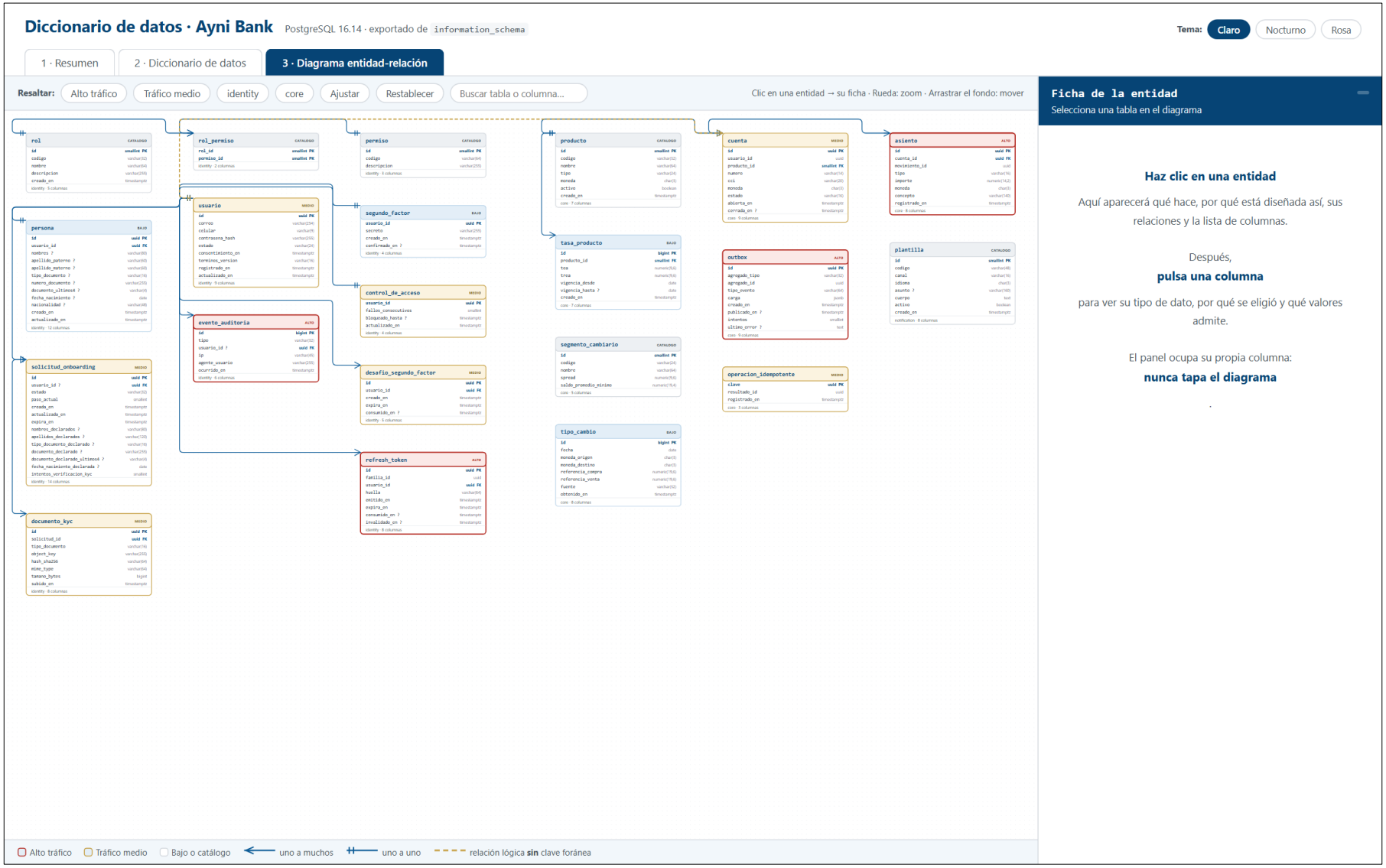



Figura 8 · El diagrama del APF1, para comparar con la figura 4.

## Fuentes consultadas


- PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*: Explicit Locking, Constraints.
- Amazon Web Services. *Amazon RDS User Guide*: Multi-AZ deployments, Read replicas, Point-in-time recovery.
- Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley (patrón Repository).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Ayni Bank. ADR-0029 · Partida doble con otros bancos (repositorio, docs/arquitectura/adr).

# 3. MEDIDAS DE SEGURIDAD DE LA APLICACIÓN

La rúbrica pide medidas que garanticen confidencialidad, integridad y disponibilidad. Las organizamos en ese mismo orden y, sobre todo, las **comprobamos contra producción**: un control que no se ha probado es una intención.

## 3.1 Módulo de autenticación y autorización

La autenticación responde a la pregunta «¿quién eres?» y la autorización a «¿qué puedes hacer?». En una banca digital las dos son críticas: si alguien suplanta a un cliente puede mover su dinero. Por eso el ingreso combina varias capas, de modo que romper una sola no basta:

- **Algo que sabes:** la contraseña, guardada con Argon2id y nunca en claro.
- **Algo que tienes:** el celular con la app autenticadora, o el correo del titular.
- **Un token de vida corta:** quince minutos, validado en el gateway antes de llegar a cualquier servicio.
- **Defensas contra abuso:** pausa del ingreso tras intentos fallidos y detección de tokens reutilizados.

La tabla detalla cada mecanismo y cómo está implementado:

**Tabla 12** · Mecanismos de autenticación y autorización

| Mecanismo | Implementación |
|---|---|
| Credenciales | Contraseña con **Argon2id** (resistente a GPU); política de contraseña validada en el dominio. |
| Segundo factor | A elección: app autenticadora (TOTP, código cada 30 s) o código por correo real con Amazon SES. |
| Tokens | JWT de acceso de 15 minutos, solo en memoria del navegador; renovación de 7 días en cookie HttpOnly, rotada en cada uso. Reutilizar un token ya usado invalida toda su familia (señal de robo). |
| Validación centralizada | El **gateway** valida firma y caducidad antes de enrutar y borra cualquier cabecera de identidad que traiga el cliente. |
| Autorización (RBAC) | 5 roles y 12 permisos sembrados; la identidad del titular sale solo del token, nunca de la URL: no se puede pedir la cuenta de otro. |
| Bloqueo | Bloqueo progresivo tras intentos fallidos y aviso por correo al titular. |
| Cierre de sesión | DELETE /api/v1/sesion invalida la familia de tokens. |

*Nota.* Elaboración propia a partir del código de ayni-identity-service y ayni-gateway.

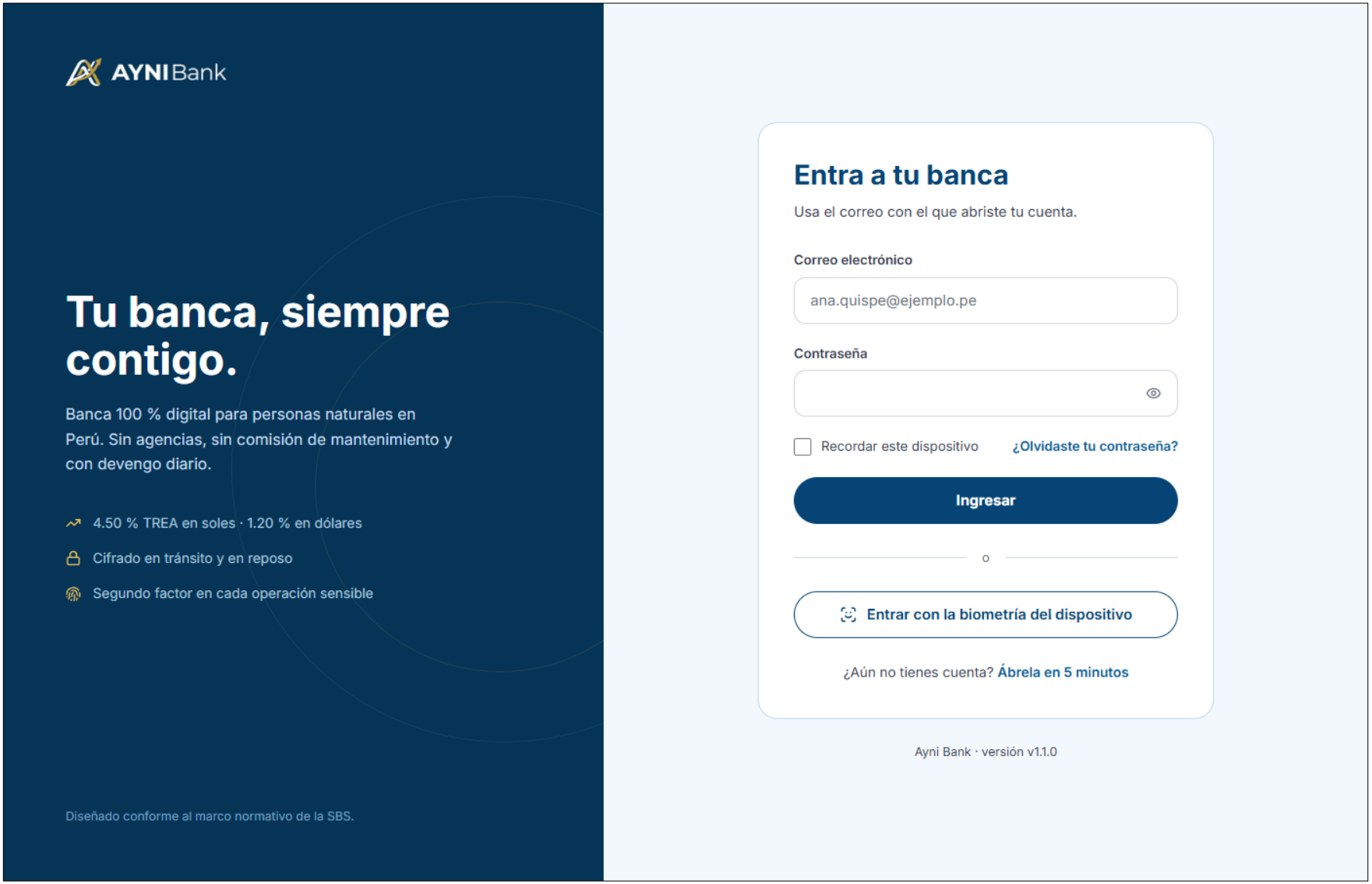



Figura 9 · Formulario de ingreso en producción.

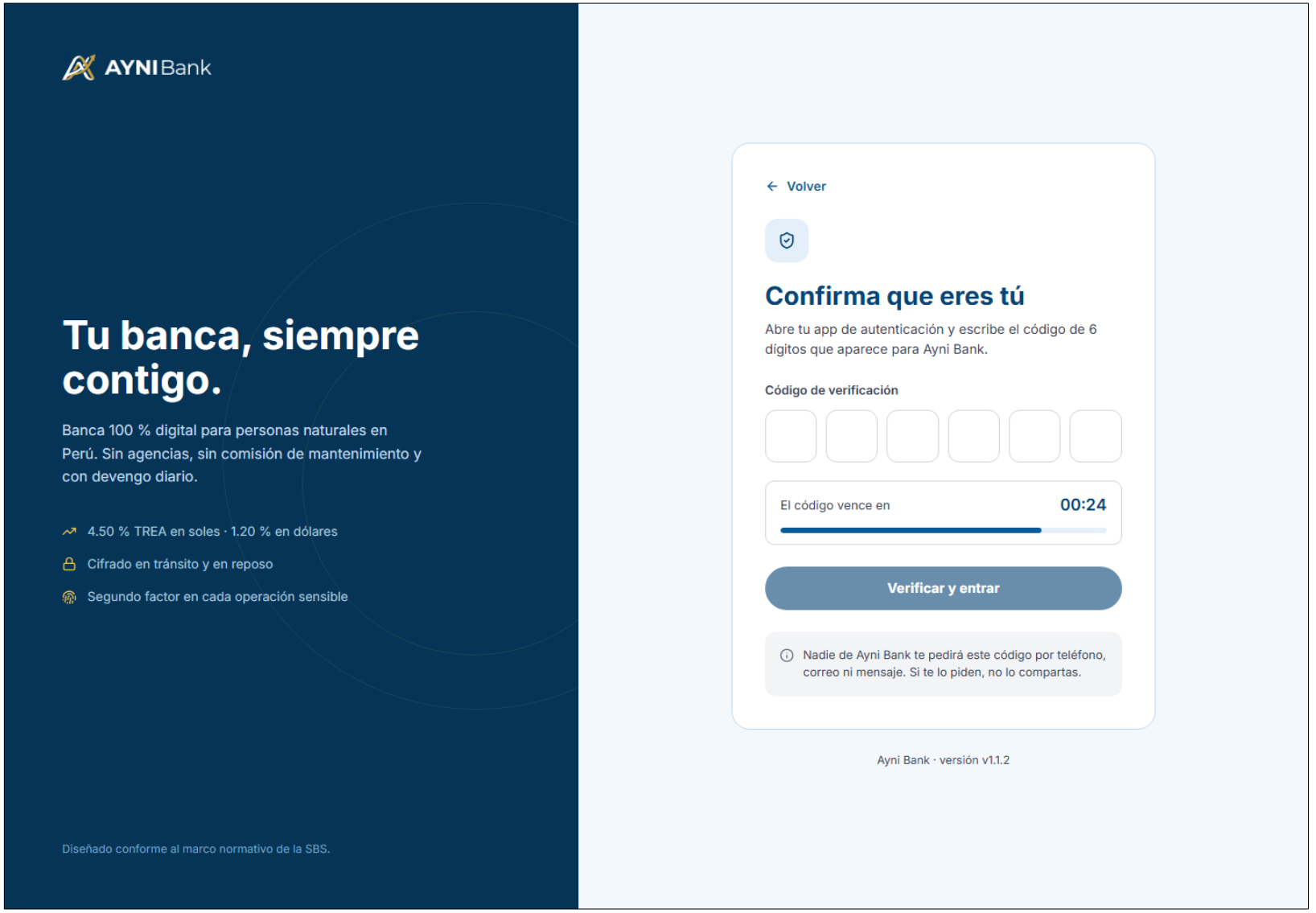



Figura 10 · Segundo factor: código de la app autenticadora.

```
gateway · FiltroDeAutenticacion.java
if (protegida(ruta)) {
    Optional<String> usuario = verificador.usuarioDe(cabecera);  // firma + caducidad
    if (usuario.isEmpty()) return responder401(intercambio);
    peticion = peticion.mutate()
        .headers(h -> h.remove(CABECERA_USUARIO))                 // no se puede falsificar
        .header(CABECERA_USUARIO, usuario.get()).build();
}
```

Código fuente: https://github.com/LOAD-13/ayni-bank/tree/main/services/ayni-identity-service y https://github.com/LOAD-13/ayni-bank/tree/main/services/ayni-gateway.

## 3.2 Informe técnico de seguridad y cifrado de datos

Este informe sigue los tres pilares clásicos de la seguridad de la información, la tríada CIA:

- **Confidencialidad:** que solo quien debe pueda leer los datos (cifrado en tránsito y en reposo).
- **Integridad:** que los datos no se alteren sin dejar rastro (partida doble, idempotencia y auditoría).
- **Disponibilidad:** que el sistema siga atendiendo y se pueda recuperar ante un fallo (monitoreo, alarmas y copias).

### 3.2.1 Confidencialidad: cifrado

Los datos viajan cifrados en cada tramo, desde el navegador hasta la base de datos, y se guardan cifrados en reposo. El dato más sensible que tratamos, el número de DNI que lee el OCR, tiene además un cifrado propio en la aplicación, de modo que ni con acceso a la base se puede leer en claro:

**Tabla 13** · Cifrado de los datos en tránsito y en reposo

| Dato o canal | Mecanismo | Estándar |
|---|---|---|
| Navegador ↔ servidor | TLS 1.2/1.3 con Let's Encrypt; HSTS de un año | NIST SP 800-52 |
| Servidor ↔ base de datos | TLS obligatorio hasta RDS | OWASP ASVS V9 |
| Datos en reposo | Cifrado KMS en RDS y en el bucket S3 de documentos | ISO 27001 A.8.24 |
| Contraseñas | Argon2id | OWASP Password Storage |
| Secreto TOTP y número de DNI | AES-256-GCM con clave en Parameter Store | NIST SP 800-38D |
| Tokens | JWT HS256 de 256 bits; renovación guardada como huella SHA-256 | RFC 7519 |
| Secretos de infraestructura | Parameter Store y Secrets Manager; nunca en el repositorio | OWASP Secrets Mgmt |

*Nota.* Elaboración propia. Los estándares citados corresponden a NIST, OWASP e ISO/IEC 27001.

### 3.2.2 Integridad y trazabilidad

La integridad del dinero la dan la partida doble, la idempotencia y el bloqueo ordenado (capítulo 2). La trazabilidad, tres mecanismos complementarios: la pista de auditoría en tabla de identity (ingresos, fallos, bloqueos, reutilización de tokens) y de core (operaciones y rechazos); los registros JSON del proxy; y métricas de negocio en Prometheus que no llevan ningún dato del cliente.

### 3.2.3 Disponibilidad

Alarmas de CloudWatch con aviso por correo (SNS), presupuesto con alerta, pruebas de humo y rollback automático en cada despliegue, y copias de la base con recuperación a un punto en el tiempo.

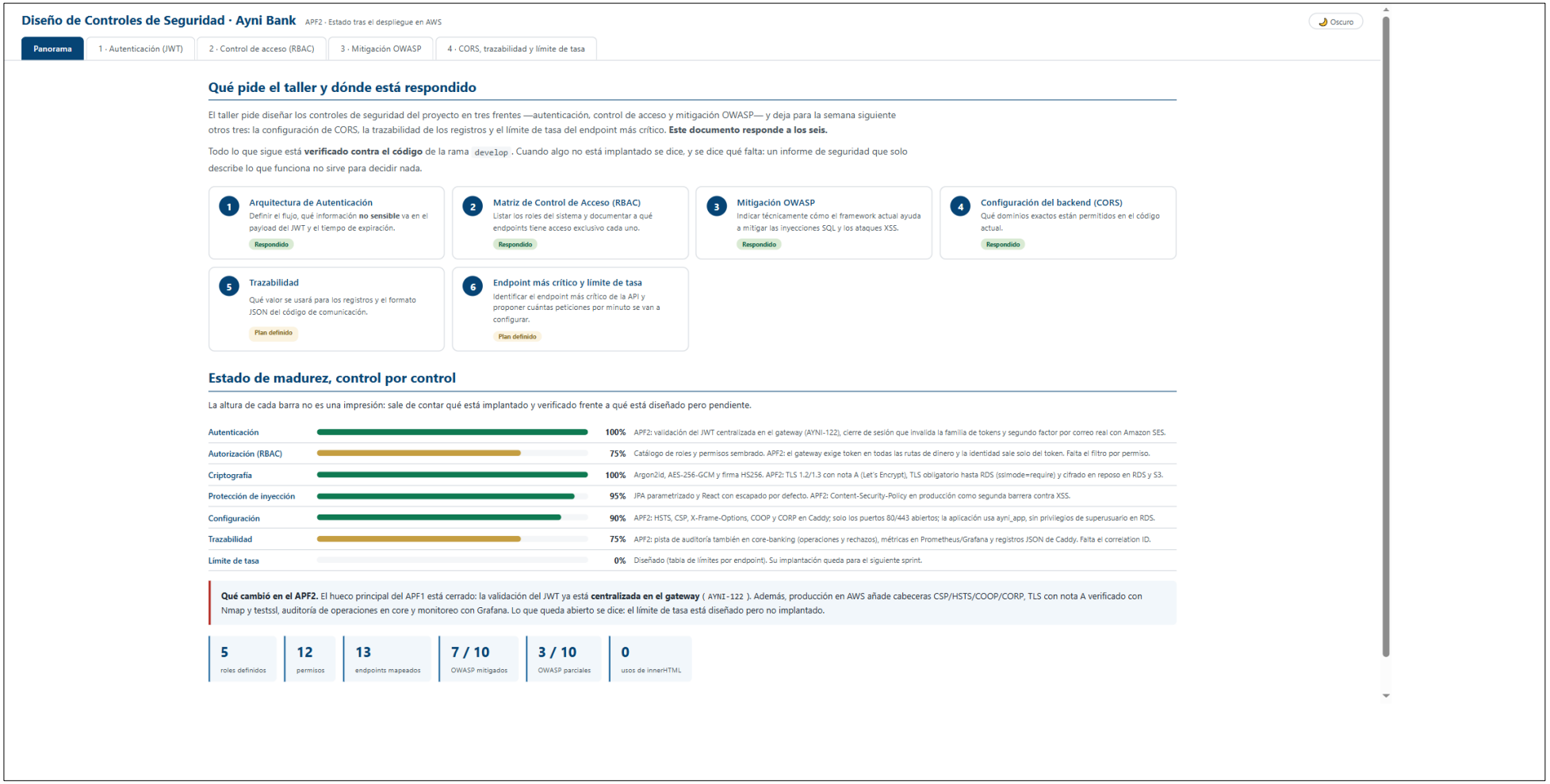



Figura 11 · Madurez de los controles tras el APF2 (antes: autenticación 90 %, configuración 55 %, trazabilidad 40 %).

## 3.3 Prueba de seguridad web

Un workflow de GitHub Actions ejecuta contra la URL de producción cuatro herramientas de la distribución Kali Linux: **OWASP ZAP** (análisis dinámico de la aplicación), **Nmap** (puertos y cifrados), **Nikto** (servidor web) y **testssl.sh** (configuración TLS). Corrimos el escaneo, corregimos lo encontrado y volvimos a escanear.




Figura 12 · Resultado del primer escaneo en producción.

**Tabla 14** · Hallazgos del escaneo de seguridad y su mitigación

| Hallazgo | Herramienta | Mitigación |
|---|---|---|
| Falta Content-Security-Policy (medio) | ZAP, Nikto | CSP en el proxy: solo origen propio, subida prefirmada a S3 y frame-ancestors 'none' |
| Falta COOP / CORP (bajo) | ZAP | Cross-Origin-Opener/Resource-Policy: same-origin |
| Cabeceras x-nextjs-* (información) | Nikto | Eliminadas en el proxy |
| CSP con 'unsafe-inline' (medio, aparece al añadir la CSP) | ZAP | Aceptado temporalmente: lo exige la hidratación de Next.js; se retirará con nonces por petición |
| Puertos expuestos | Nmap | Solo 80 y 443; base de datos y servicios sin acceso público |
| Configuración TLS | testssl, Nmap | Nota A; sin SSLv3 ni TLS 1.0/1.1; 13 de 14 pruebas sin vulnerabilidad |
| Inyección SQL y XSS | Revisión de código | JPA parametrizado; React escapa por defecto; cero usos de innerHTML |
| CORS | Configuración | Lista blanca con el dominio de producción, nunca «*» |

*Nota.* Elaboración propia a partir de los reportes de OWASP ZAP, Nmap, Nikto y testssl.sh ejecutados contra producción.

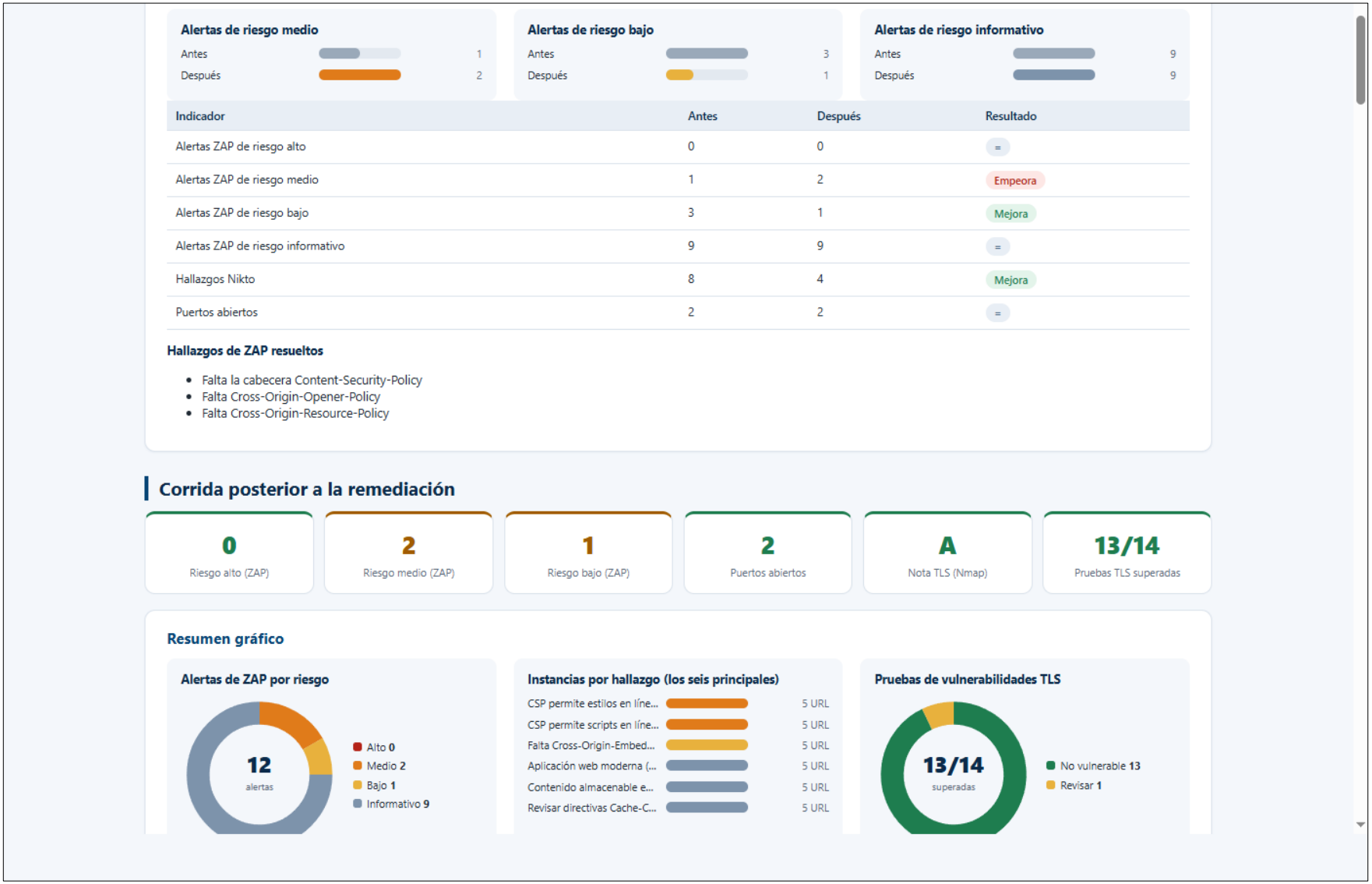



Figura 13 · Comparativa antes y después de la remediación.

## 3.4 Catálogo de controles de seguridad

**Tabla 15** · Catálogo de controles de seguridad

| ID | Control | Referencia | Estado |
|---|---|---|---|
| CS-01 | Autenticación multifactor | ISO 27001 A.8.5 · OWASP A07 | Implantado |
| CS-02 | Almacenamiento seguro de contraseñas (Argon2id) | OWASP ASVS V2 | Implantado |
| CS-03 | Gestión de sesión con rotación de tokens | OWASP ASVS V3 | Implantado |
| CS-04 | Validación central del JWT en el gateway | NIST SP 800-207 | Implantado |
| CS-05 | Control de acceso por roles (RBAC) | ISO 27001 A.5.15 · OWASP A01 | Parcial |
| CS-06 | Cifrado en tránsito TLS 1.2+ | ISO 27001 A.8.24 | Implantado |
| CS-07 | Cifrado en reposo (KMS, AES-256-GCM) | ISO 27001 A.8.24 | Implantado |
| CS-08 | Consultas parametrizadas (anti-SQLi) | OWASP A03 | Implantado |
| CS-09 | Cabeceras de seguridad (CSP, HSTS, XFO) | OWASP A05 | Implantado |
| CS-10 | Secretos fuera del código | ISO 27001 A.8.4 | Implantado |
| CS-11 | Escaneo de dependencias, imágenes y secretos (Trivy, gitleaks) | OWASP A06 | Implantado |
| CS-12 | Pista de auditoría en base de datos | ISO 27001 A.8.15 | Implantado |
| CS-13 | Monitoreo y alertas | ISO 27001 A.8.16 | Implantado |
| CS-14 | Mínimo privilegio en base de datos y nube (IAM, ayni_app) | ISO 27001 A.8.2 | Implantado |
| CS-15 | Respaldo y recuperación a un punto en el tiempo | ISO 27001 A.8.13 | Implantado |
| CS-16 | Pruebas de seguridad automatizadas | OWASP WSTG | Implantado |

*Nota.* Elaboración propia con referencia a ISO/IEC 27001:2022 (Anexo A), OWASP ASVS y NIST.

El control CS-05 figura como parcial a propósito: el catálogo de roles y permisos existe y el gateway exige token en todas las rutas de dinero, pero falta el filtro que compruebe cada permiso por endpoint. Declararlo es más útil que presentarlo como terminado.

### Fuentes consultadas


- OWASP Foundation. *OWASP Top 10:2021* y *Application Security Verification Standard 4.0.3*.
- ISO/IEC 27001:2022, Anexo A.
- NIST SP 800-63B (autenticación) y SP 800-38D (GCM).
- Congreso de la República del Perú. Ley N.º 29733, de Protección de Datos Personales.

# 4. DESPLIEGUE DE LA APLICACIÓN EN LA NUBE

Este capítulo demuestra que el sistema no solo funciona en el computador de un desarrollador, sino en la nube, accesible para cualquier cliente y protegido. Sigue el orden natural de una puesta en producción:

- **Plan de pruebas:** qué se prueba, cómo y con qué criterio se da por bueno.
- **Manual de despliegue:** cómo se crea la infraestructura y cómo llega cada versión a producción.
- **Evidencia:** el pipeline en verde, las pruebas de humo y los incidentes reales con su respuesta.
- **Monitoreo:** cómo vigilamos la base de datos y la aplicación una vez en producción.

## 4.1 Plan de pruebas del sistema

El plan de pruebas define qué se verifica antes de dar por buena una versión. Combina tres niveles, del más rápido al más cercano a un cliente real:

- **Pruebas automáticas en cada pull request:** unitarias, de integración, de arquitectura y de interfaz; si una falla, el cambio no se puede fusionar.
- **Pruebas de humo tras cada despliegue:** diez comprobaciones contra la URL pública; si una falla, el pipeline revierte la versión.
- **Validación manual:** el recorrido de un cliente real en producción y el de las operaciones completas en el entorno local.

La primera tabla resume el plan y la segunda los casos de prueba del sistema con su resultado:

**Tabla 16** · Plan de pruebas del sistema

| Aspecto | Definición |
|---|---|
| Alcance | HU-01 registro, HU-02 DNI, HU-03 selfie, HU-04 ingreso con 2FA, HU-05 cuenta, HU-07 transferencia y HU-08 saldo; y los no funcionales de seguridad, accesibilidad y despliegue. |
| Enfoque | Pirámide de pruebas automatizada en cada PR; pruebas de humo tras cada despliegue; validación manual en producción y en local. |
| Recursos | GitHub Actions, JUnit 5, Mockito, Cucumber, ArchUnit, pytest, Vitest, Playwright, OWASP ZAP, Nmap, Nikto y testssl. |
| Criterio de salida | Cero pruebas fallidas, cobertura del dominio ≥ 80 % (SonarCloud), cero vulnerabilidades altas y pruebas de humo en verde. |
| Cronograma | Continuo en cada PR del Sprint 2; escaneo de seguridad a demanda; validación en producción en cada release. |

*Nota.* Elaboración propia.

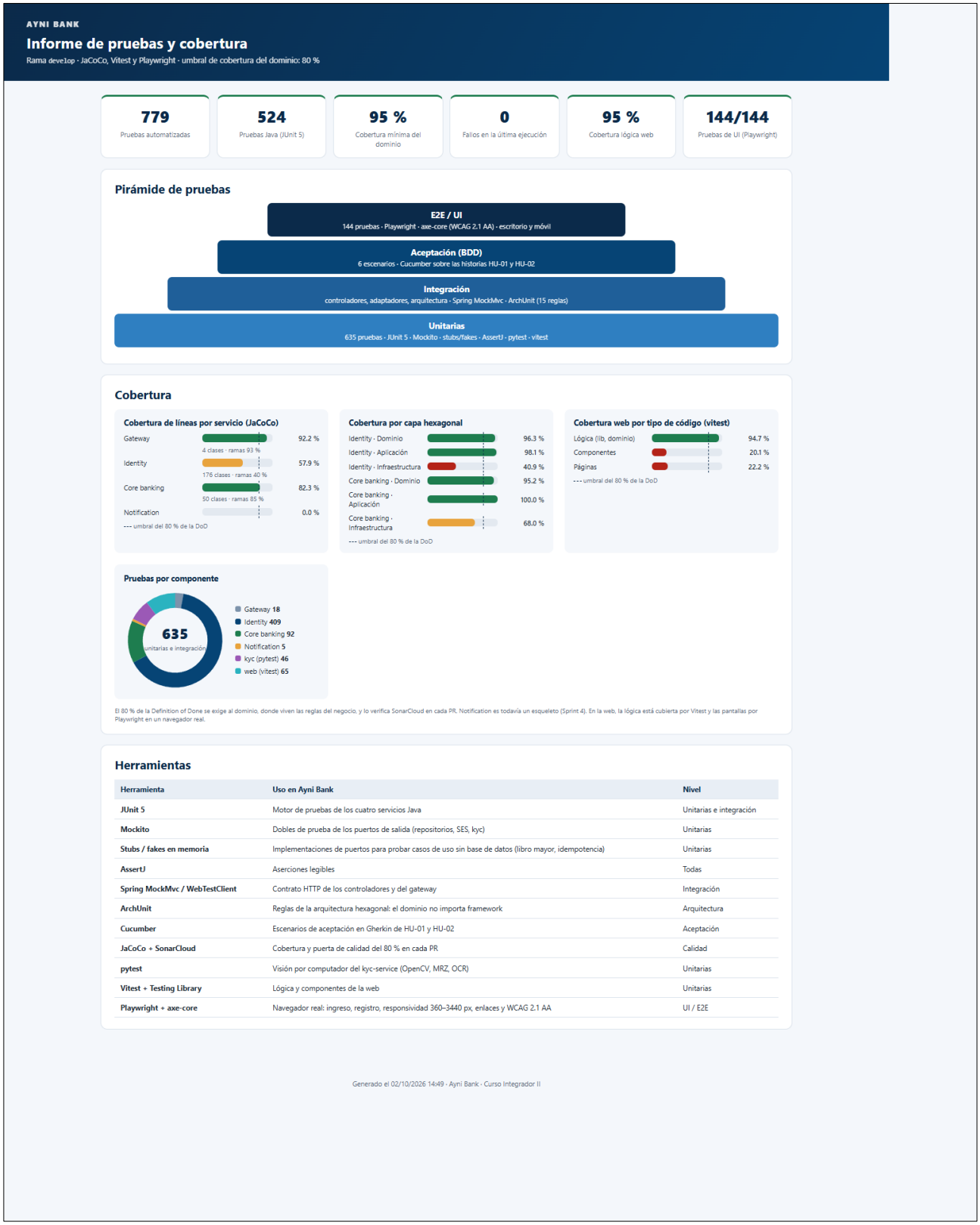



Figura 14 · Pirámide de pruebas y cobertura (JaCoCo, Vitest, Playwright).

**Tabla 17** · Casos de prueba del sistema y sus resultados

| ID | Req. | Caso | Resultado esperado | Estado |
|---|---|---|---|---|
| CP-01 | HU-01 | Registro con datos válidos | Llega el correo de continuación | Aprobado |
| CP-02 | HU-02 | Foto del DNI legible | El OCR extrae los datos y el titular los confirma | Aprobado |
| CP-03 | HU-04 | Ingreso con contraseña y 2FA | Acceso al panel y evento INGRESO_EXITOSO | Aprobado |
| CP-04 | HU-04 | Contraseña errónea repetida | Ingreso bloqueado y aviso por correo | Aprobado |
| CP-05 | HU-08 | Consultar saldo | Saldo = suma de asientos | Aprobado |
| CP-06 | HU-07 | Depósito de S/ 1 500 | Dos asientos y comprobante | Aprobado |
| CP-07 | HU-07 | Transferencia de S/ 250 a otra cuenta Ayni | Cargo, abono y comprobante | Aprobado |
| CP-08 | HU-07 | Transferencia por encima del límite | Rechazo con su motivo y evento de auditoría | Aprobado |
| CP-09 | HU-07 | Misma clave de idempotencia dos veces | Un solo movimiento | Aprobado |
| CP-10 | Seg. | Ruta protegida sin token | 401 en el gateway | Aprobado |
| CP-11 | Seg. | Cabeceras de seguridad | HSTS, CSP y X-Frame-Options presentes | Aprobado |
| CP-12 | HU-17 | Accesibilidad WCAG 2.1 AA | Cero violaciones (axe-core) | Aprobado |
| CP-13 | HU-20 | Despliegue con prueba de humo fallida | Rollback automático | Aprobado |

*Nota.* Elaboración propia. «Req.» indica la historia de usuario o el requisito que valida cada caso.

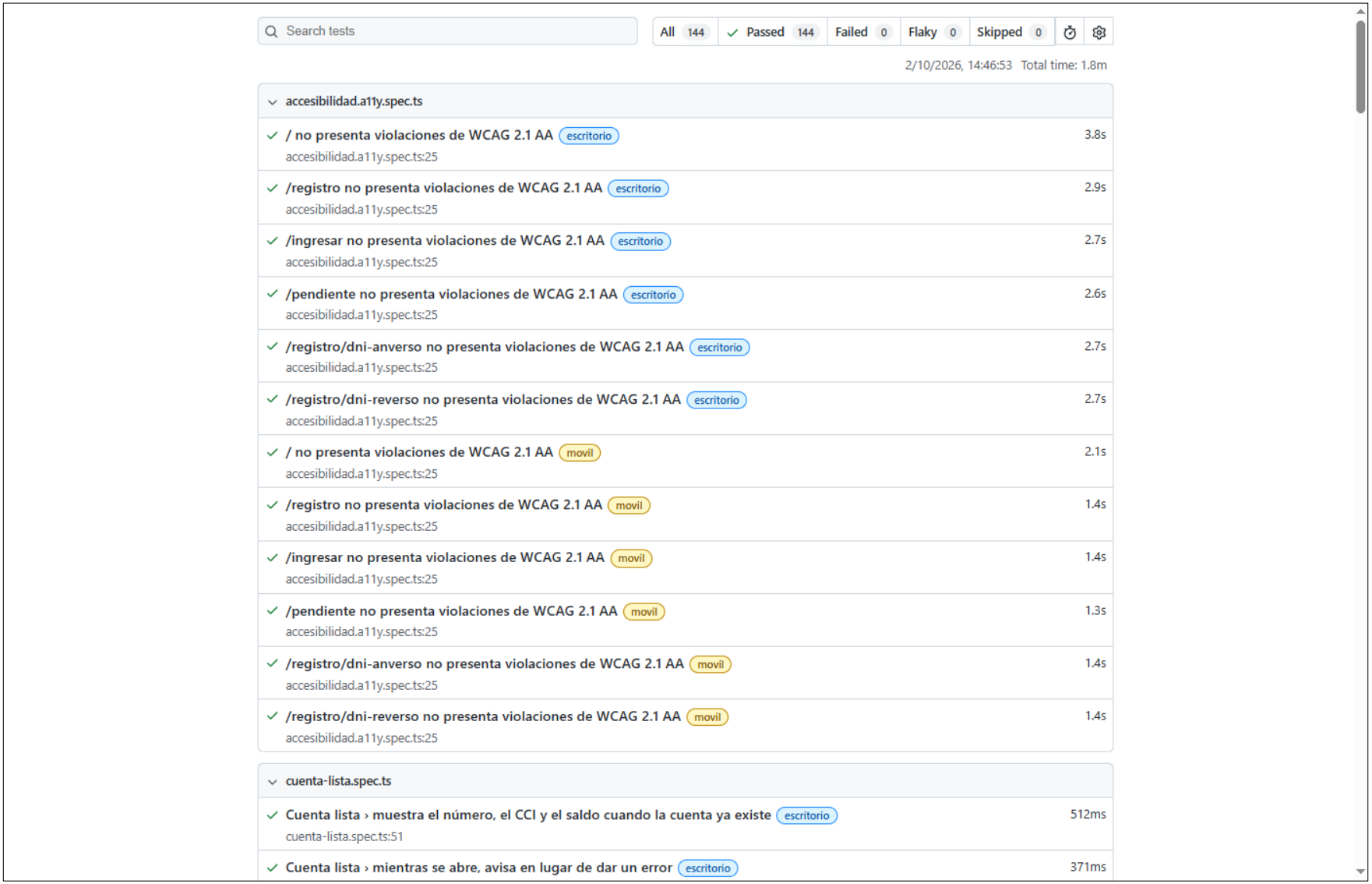



Figura 15 · Pruebas de interfaz con Playwright: 144 de 144, escritorio y móvil.

## 4.2 Manual de despliegue

**Entorno: Amazon Web Services, región us-east-1, plan gratuito.** Toda la infraestructura se describe como código en una plantilla de CloudFormation; el manual completo está en el repositorio (`docs/despliegue/manual-de-despliegue.md`). El costo es cero: la cuenta usa créditos del plan gratuito y un presupuesto con alarma avisa antes de cualquier cargo.

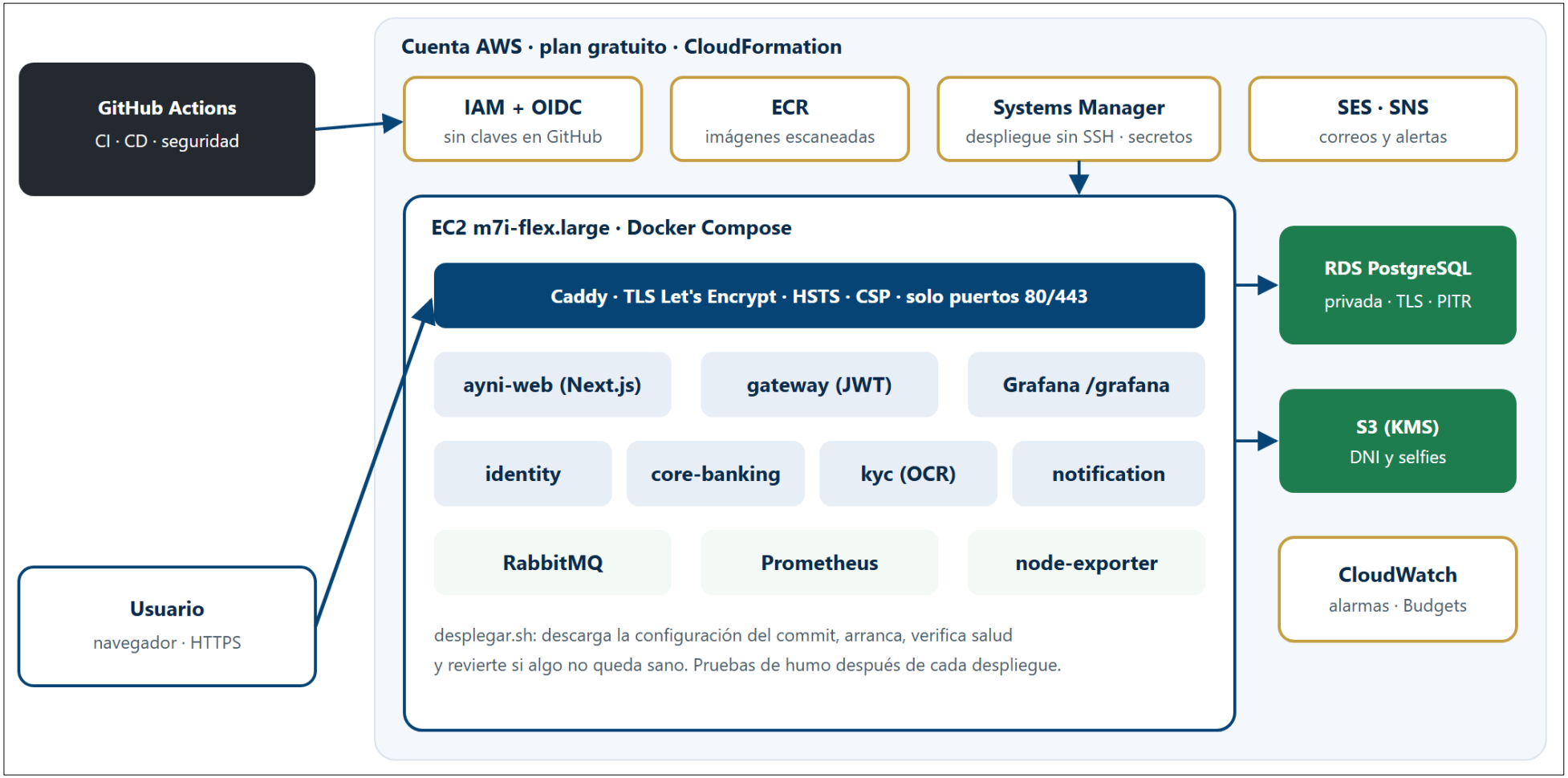



Figura 16 · Arquitectura de despliegue en AWS.

**Tabla 18** · Componentes de la infraestructura en AWS

| Componente | Servicio AWS | Función |
|---|---|---|
| Cómputo | EC2 m7i-flex.large (8 GB) | Docker Compose con los servicios, Caddy, Prometheus y Grafana |
| Base de datos | RDS PostgreSQL 17 | Datos de identity, core y notification |
| Documentos | S3 cifrado | Fotos del DNI y selfies, con URL prefirmada |
| Imágenes | ECR | Imágenes construidas y escaneadas por el pipeline |
| Operación | Systems Manager | Despliegue sin SSH; parámetros y secretos |
| Correo | SES | Correos de registro y de seguridad |
| Monitoreo | CloudWatch, SNS, Budgets | Alarmas, avisos al equipo y control de costo |
| Identidad del pipeline | IAM + OIDC de GitHub | Sin claves de acceso guardadas en GitHub |

*Nota.* Elaboración propia a partir de la plantilla de CloudFormation del repositorio.

**Tabla 19** · Pasos del manual de despliegue

| Paso | Qué | Cómo |
|---|---|---|
| 1 | Crear la infraestructura | `aws cloudformation deploy --template-file infra/aws/ayni-bank.cfn.yaml --stack-name ayni-bank-prod` |
| 2 | Cargar los secretos | Parameter Store: claves JWT y de cifrado, contraseñas de base, RabbitMQ y Grafana |
| 3 | Conectar GitHub | Variables AWS_ROL_DESPLIEGUE, AWS_INSTANCIA y AYNI_URL; el rol se asume por OIDC |
| 4 | Desplegar | Fusionar un PR a `main`: el workflow `cd.yml` hace el resto |
| 5 | Pipeline | CI completa → imágenes a ECR → Systems Manager ejecuta desplegar.sh → pruebas de humo → release |
| 6 | Revertir | Automático si una prueba de humo falla; manual relanzando el workflow con la etiqueta anterior |
| 7 | Verificar | `curl https://ayni.produccion.ejemplo/api/health` → UP y la versión bajo el formulario de ingreso |

*Nota.* Elaboración propia. El manual completo se encuentra en el repositorio del proyecto.

## 4.3 Evidencia de pruebas de despliegue

La evidencia más fuerte de que el despliegue es fiable es el propio pipeline. Cada vez que un cambio entra en la rama main, GitHub Actions ejecuta la misma secuencia sin intervención humana:

- **Integración continua completa:** las más de setecientas pruebas, la cobertura y el análisis de SonarCloud.
- **Imágenes Docker escaneadas** con Trivy y publicadas en Amazon ECR.
- **Despliegue en la EC2** con Systems Manager, sin SSH, y migraciones de la base aplicadas al arrancar.
- **Pruebas de humo** contra la URL pública y, si fallan, **reversión automática** a la versión anterior.

La figura muestra una ejecución completa en verde y la tabla, las diez pruebas de humo que se corren después de cada despliegue:

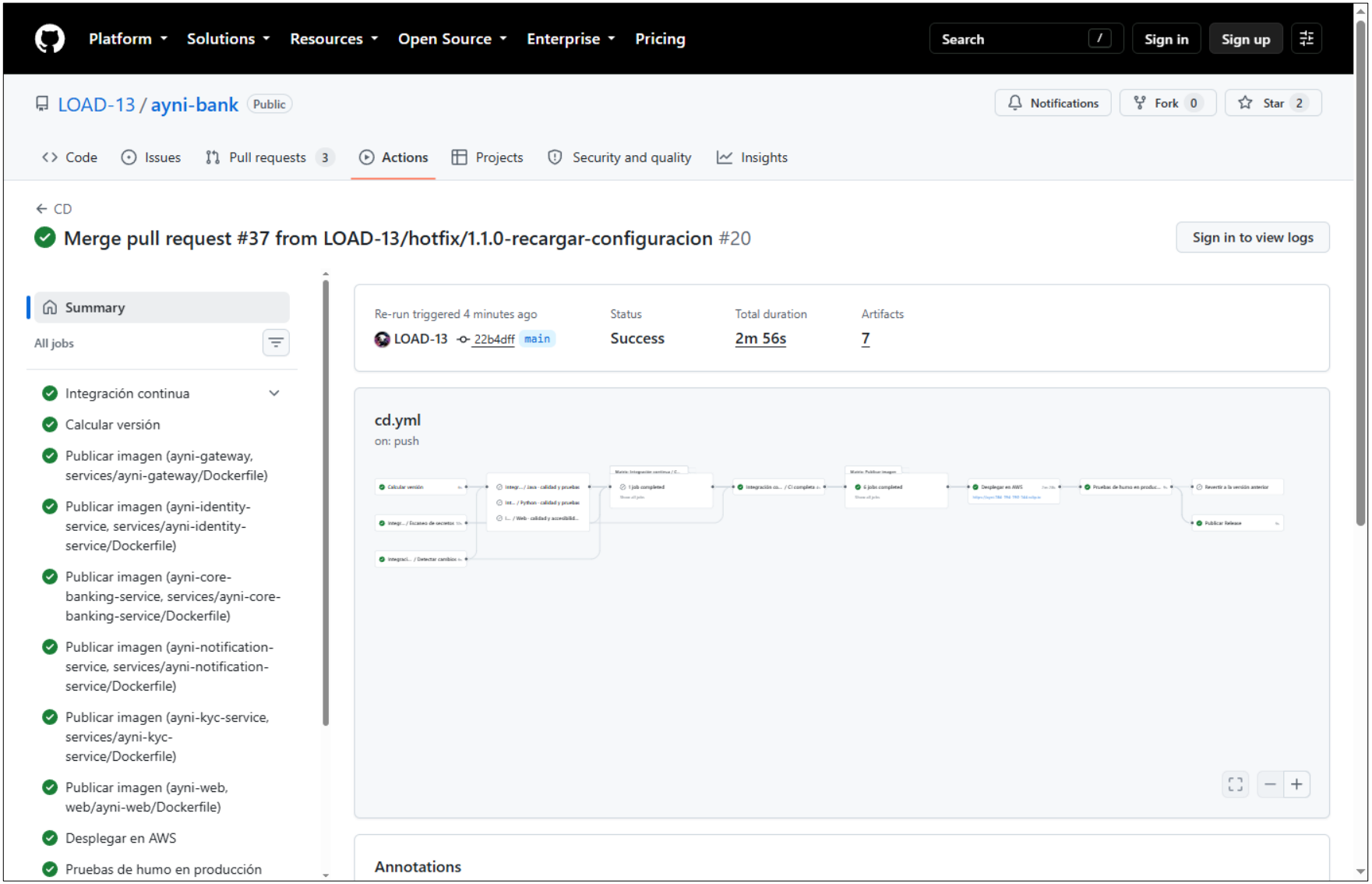



Figura 17 · Pipeline de despliegue continuo completo en verde.

**Tabla 20** · Pruebas de humo ejecutadas tras cada despliegue

| Prueba de humo (tras cada despliegue) | Resultado |
|---|---|
| GET /api/health responde UP | OK |
| La web responde 200 | OK |
| La pantalla de ingreso muestra la versión desplegada | OK |
| HTTP redirige a HTTPS | OK |
| Cabecera HSTS | OK |
| X-Frame-Options: DENY | OK |
| Content-Security-Policy | OK |
| Sin X-Powered-By | OK |
| Ingreso inválido → 400 | OK |
| Ruta protegida sin token → 401 | OK |

*Nota.* Elaboración propia a partir de las ejecuciones del pipeline en GitHub Actions.

### 4.3.1 Lo que falló y cómo respondió el sistema

Publicar la versión 1.1.0 no salió a la primera, y lo contamos porque es la mejor evidencia de que el despliegue es seguro. Un despliegue no se mide por no fallar nunca, sino por lo que pasa cuando falla: si el error llega a los clientes o si el sistema lo detecta y se protege solo. Hubo tres incidentes, y en ninguno los clientes perdieron el servicio:

**Tabla 21** · Incidentes en la publicación de la versión 1.1.0 y respuesta del sistema

| Incidente | Qué pasó | Qué hizo el sistema | Corrección |
|---|---|---|---|
| Cabecera CSP ausente | El proxy no recargó su configuración nueva | Las pruebas de humo fallaron y el pipeline **revirtió solo** a la 1.0.0 sin caída | Reinicio de los contenedores con configuración montada (PR #37) |
| Disco al 91 % | Imágenes antiguas llenaron el disco y el agente de despliegue murió | Producción siguió en la versión anterior | Limpieza de imágenes sin uso antes de descargar (PR #39) |
| Subida del DNI bloqueada | La CSP y el CORS del bucket no permitían la subida prefirmada | La web mostró un error recuperable | Origen de S3 en la CSP y POST en el CORS (PR #40) |

*Nota.* Elaboración propia a partir del historial del pipeline y de los pull requests de corrección.

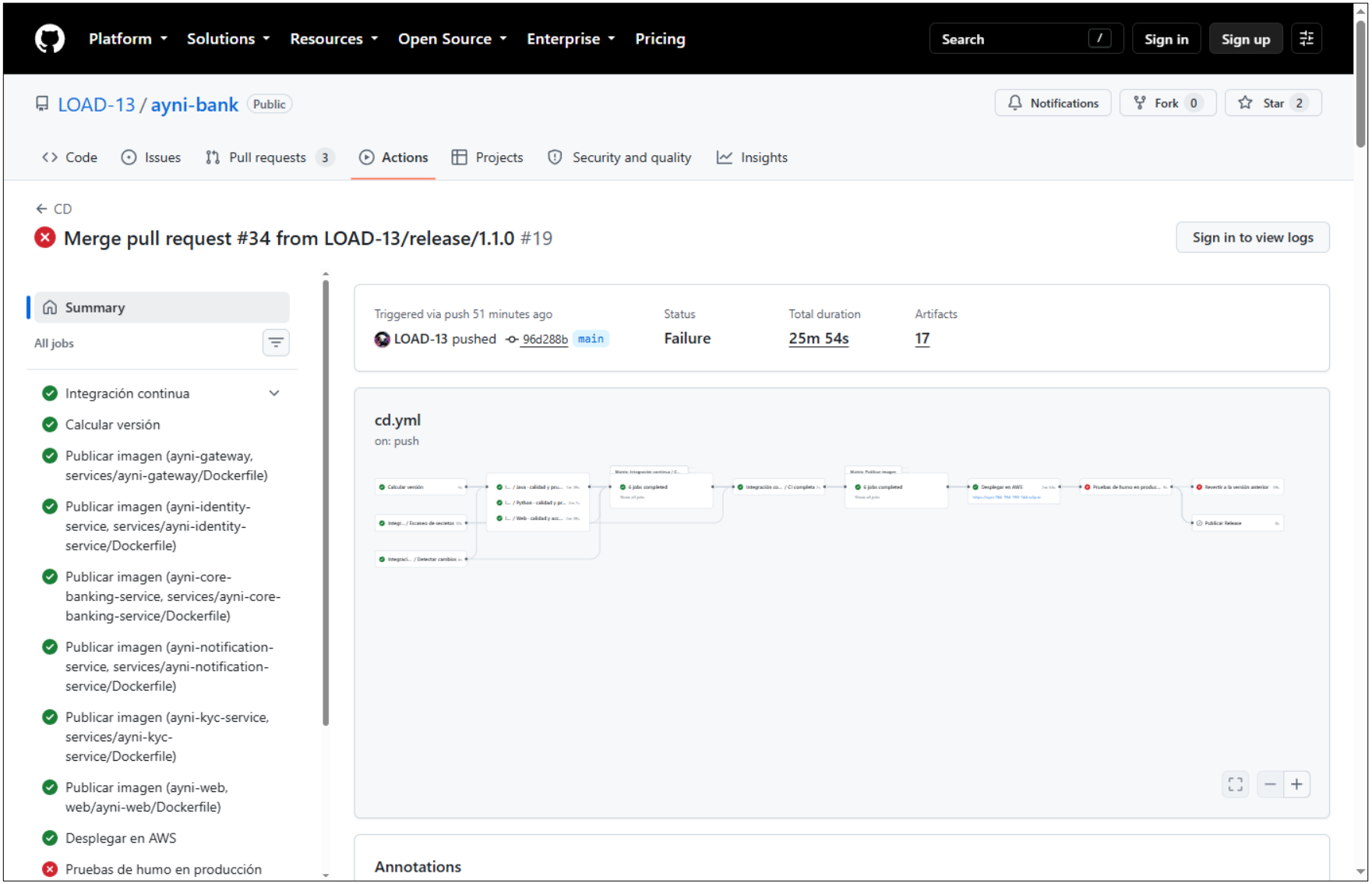



Figura 18 · Pruebas de humo fallidas y reversión automática en la 1.1.0.

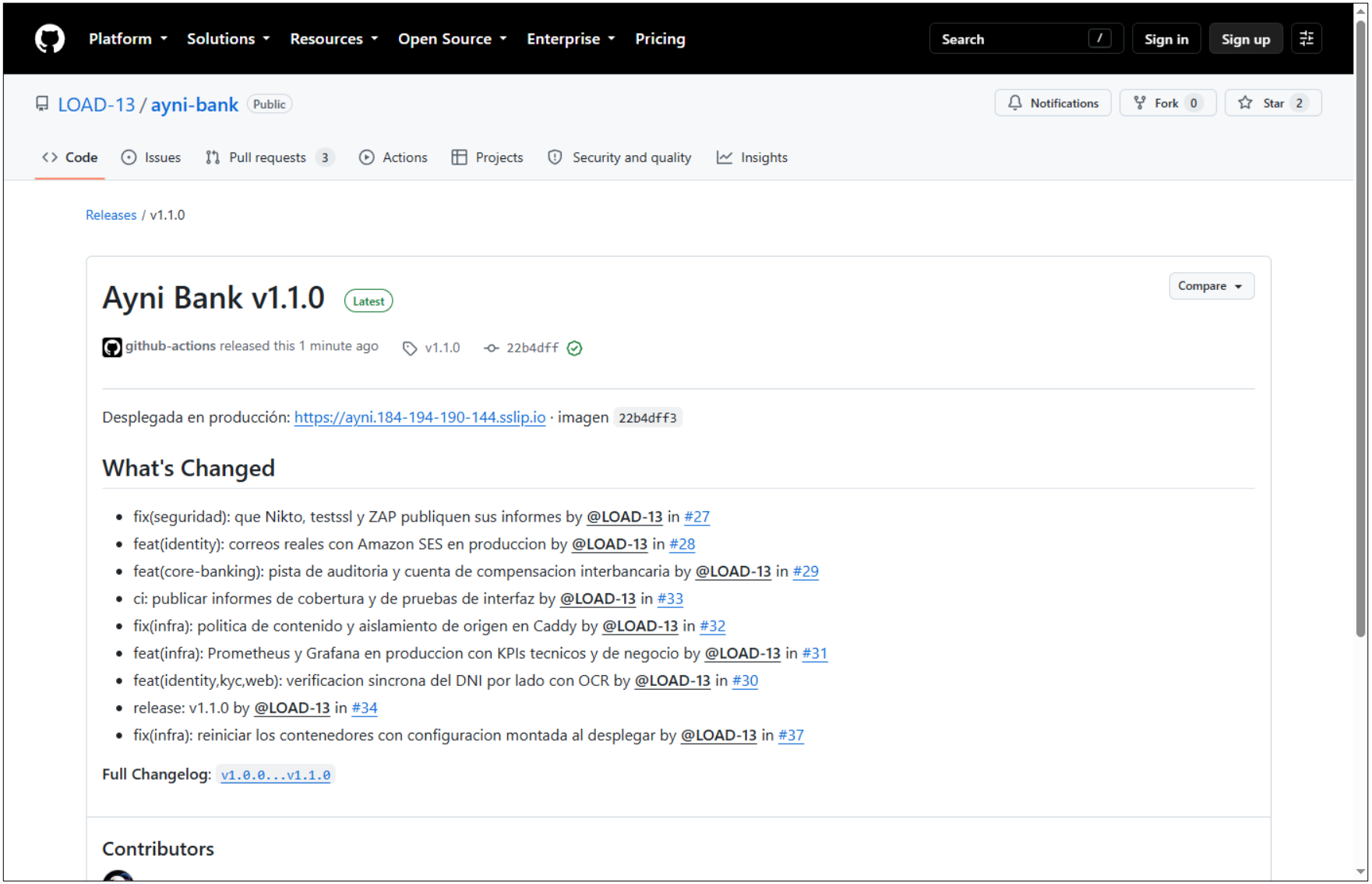



Figura 19 · Release v1.1.0 publicada por el pipeline.

### 4.3.2 De la 1.1.1 a la 1.1.6: corregir en producción con el mismo pipeline

Al usar el sistema en producción con un DNI real aparecieron fallos que las pruebas automáticas no habían visto. Cada corrección siguió el mismo camino: pull request revisado por otra integrante, integración continua en verde, release hacia main y despliegue automático con pruebas de humo. Ninguna se aplicó a mano en el servidor.

**Tabla 22** · Releases correctivas de la 1.1.2 a la 1.1.6

| Versión | Qué cambió | Por qué |
|---|---|---|
| 1.1.2 | Panel de la banca y onboarding con el diseño aprobado; limpieza automática del disco | La interfaz no seguía el prototipo y el disco llegó al 91 % |
| 1.1.3 | Reverso recortado aceptado, anverso enviado como reverso rechazado, nombres comparados por palabras | Un DNI real se rechazaba o terminaba en revisión manual |
| 1.1.4 | Lectura del MRZ también en el anverso y segundo apellido completado desde el anverso | El DNI azul lleva el MRZ en el anverso: la lectura salía «ilegible» |
| 1.1.5 | OCR con modelos ligeros (41 s → 8 s) y espera de 90 s para la extracción | La verificación quedaba «diferida» por tiempo de espera |
| 1.1.6 | Pestaña de Configuración, entrega de la selfie con aprobación por un operador y KPIs de negocio desde cero | Tras la selfie nadie aprobaba la solicitud y el tablero de KPIs no contaba la primera operación |

*Nota.* Elaboración propia a partir de las releases publicadas en GitHub.

La release 1.1.6 se grabó en video de principio a fin como evidencia del despliegue continuo: desde abrir la rama de release hasta ver la pestaña nueva en producción, sin entrar al servidor.

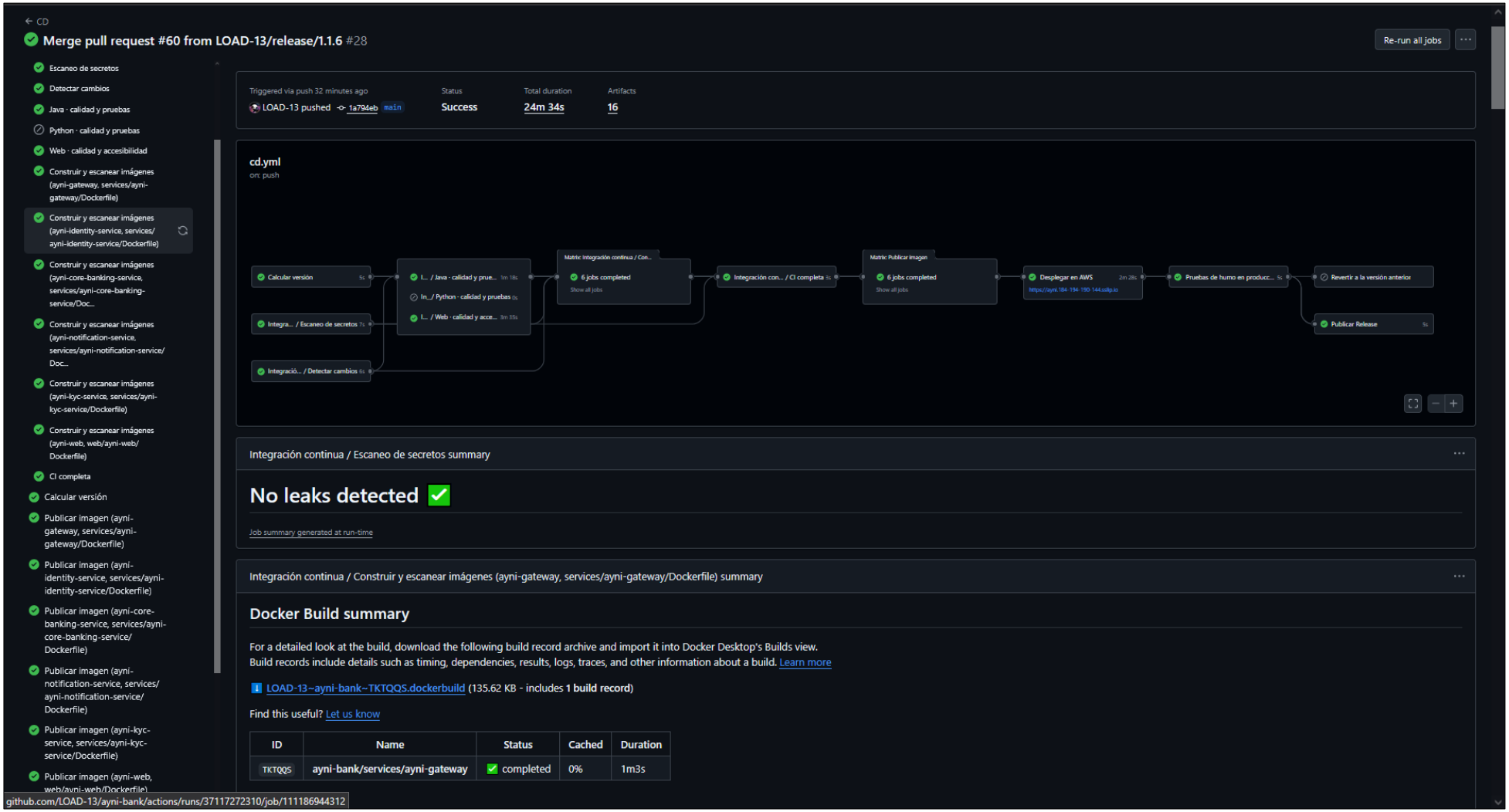



Figura 19a · Pipeline de la release 1.1.6 completo en verde (24 min 34 s), con la reversión omitida porque las pruebas de humo pasaron.

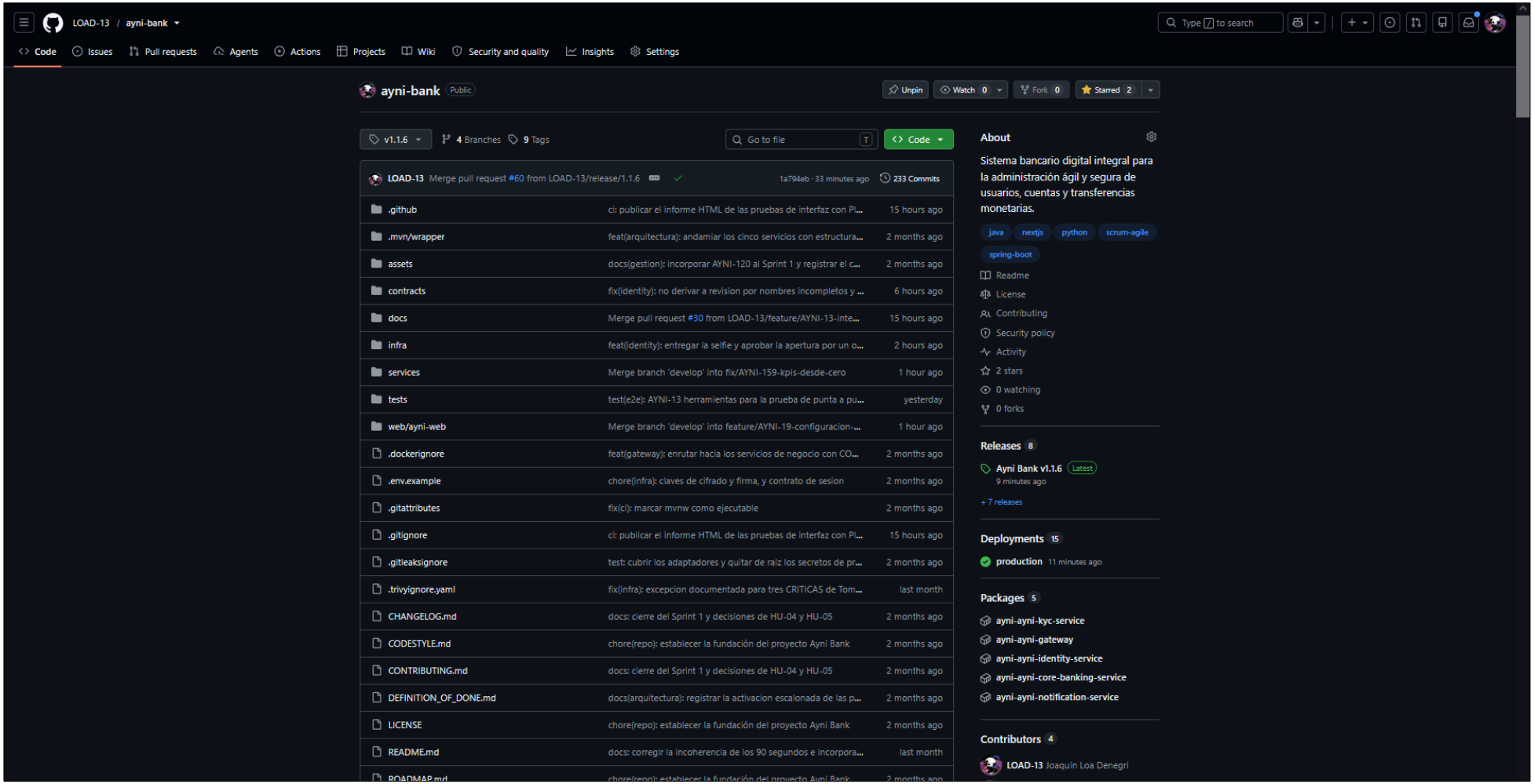



Figura 19b · Release v1.1.6 publicada por el pipeline.

### 4.3.3 Infraestructura creada en AWS

Toda la infraestructura se crea con una sola plantilla de **CloudFormation**, versionada en el repositorio: si hubiera que reconstruir producción desde cero, basta con volver a desplegar la plantilla. La aplicación corre en una instancia EC2 con una IP elástica, que es la que aparece en la URL pública; la instancia no tiene el puerto SSH abierto y se administra solo con Systems Manager.

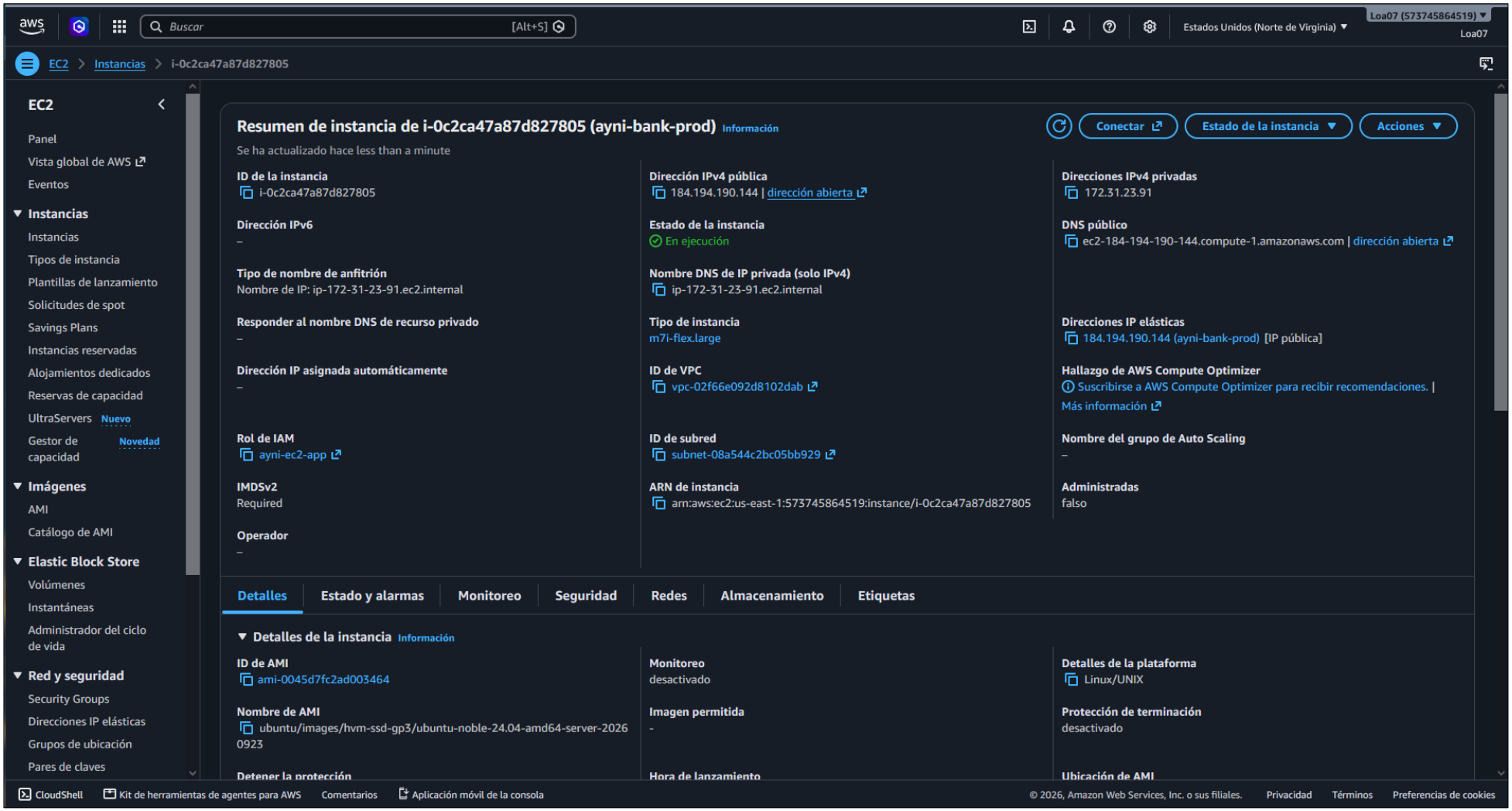



Figura 20 · Instancia EC2 de producción en la consola de AWS.

La plantilla crea, además de la instancia, la base de datos RDS, el bucket S3 de documentos KYC, los repositorios de imágenes en ECR, los roles de IAM (incluido el que usa GitHub Actions por OIDC, sin claves guardadas), los parámetros de SSM con los secretos y las alarmas de CloudWatch.

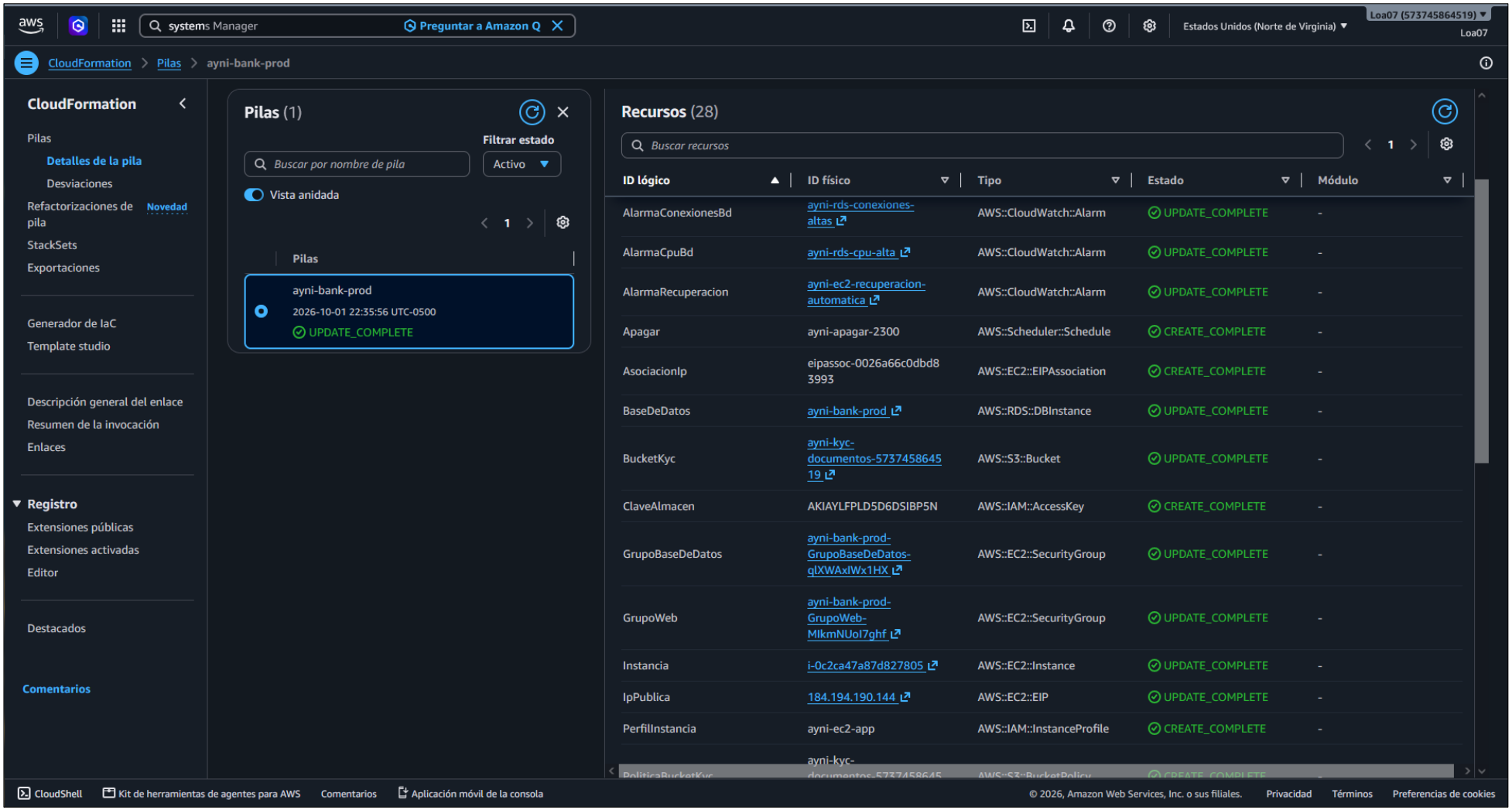



Figura 21 · Recursos creados por la plantilla de CloudFormation.

## 4.4 Monitoreo y administración de la base de datos en producción

El monitoreo tiene tres capas: las métricas de la base de datos en CloudWatch, las alarmas que avisan por correo cuando algo se sale de lo normal y Grafana, que muestra la salud de los servicios de la aplicación. Cada capa responde a una pregunta distinta: ¿la base está bien?, ¿hay que actuar ya? y ¿la aplicación atiende a los clientes?

### 4.4.1 La base de datos en Amazon RDS

La base es un PostgreSQL 17 gestionado por Amazon RDS, cifrado en reposo y sin IP pública: solo la instancia de la aplicación puede conectarse. RDS se encarga de las copias de seguridad diarias y de la recuperación a un punto en el tiempo.

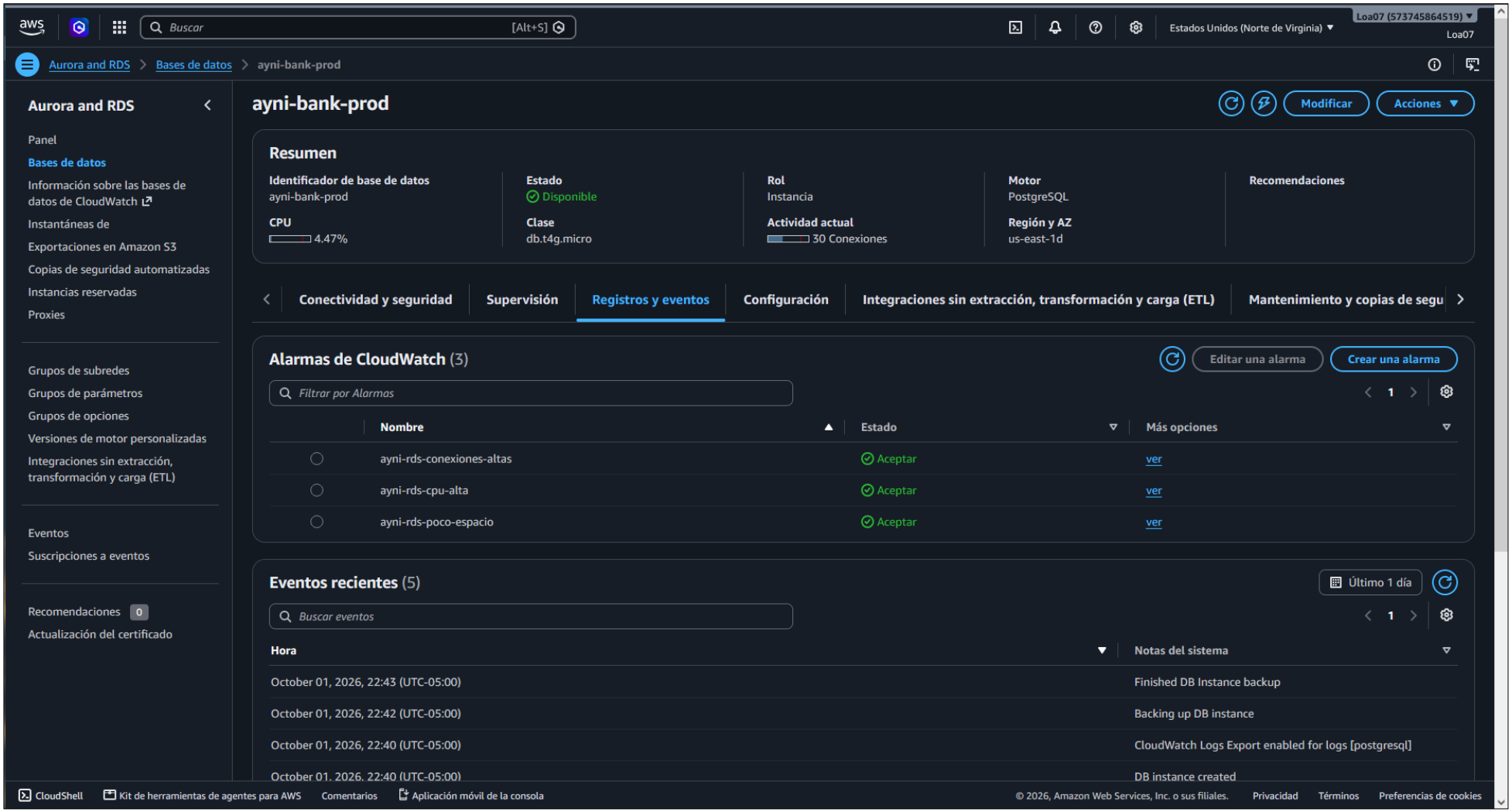



Figura 22 · Instancia RDS de producción en la consola.

Estas son las tres métricas que vigilamos. La **CPU** indica si las consultas cuestan más de lo previsto; las **conexiones** confirman que los servicios usan el pool de conexiones y no abren una por petición; el **almacenamiento libre** avisa con antelación antes de que la base se quede sin espacio.

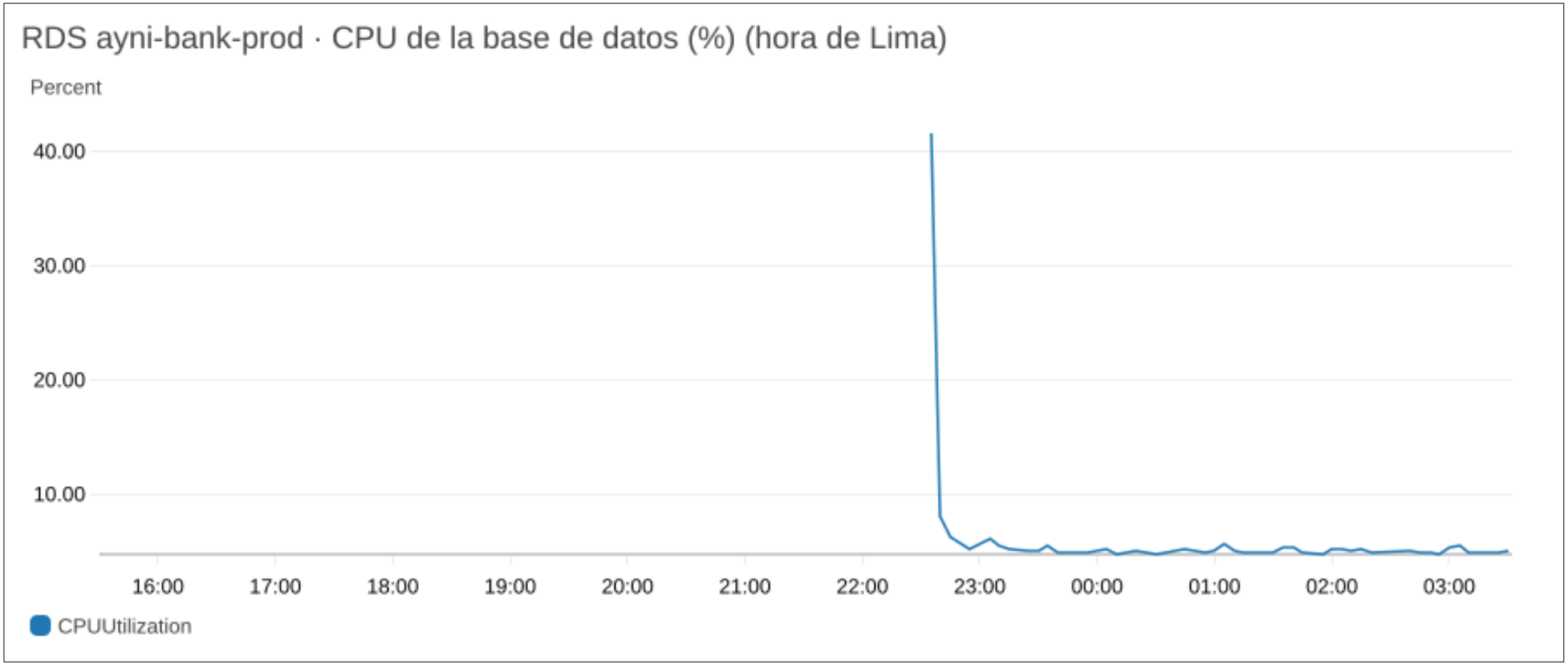



Figura · RDS en CloudWatch: uso de CPU.




Figura · RDS en CloudWatch: conexiones abiertas.

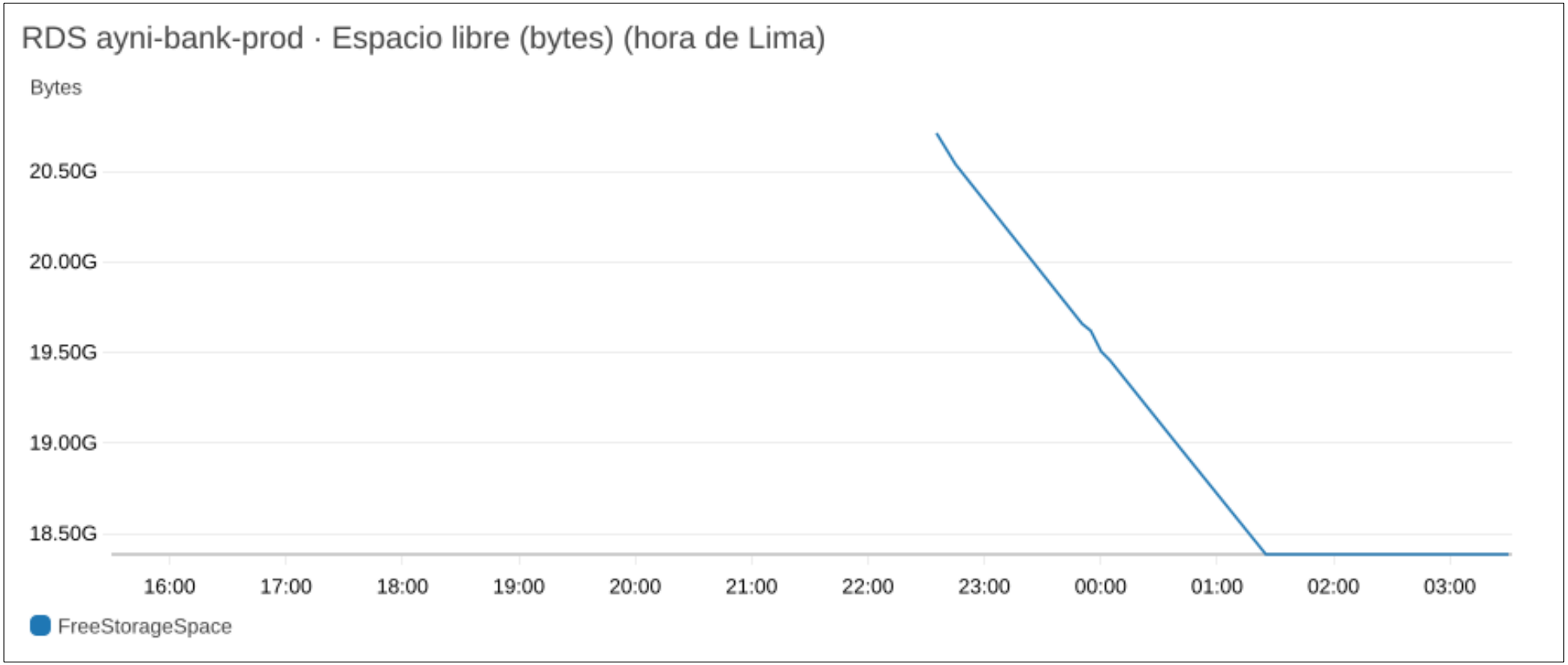



Figura · RDS en CloudWatch: almacenamiento libre.

### 4.4.2 Alarmas

Cada métrica tiene un umbral y una alarma de CloudWatch. Cuando una alarma salta, Amazon SNS envía un correo al equipo; así nos enteramos de un problema antes que los clientes. También hay una alarma de presupuesto, porque el proyecto debe mantenerse dentro del plan gratuito.

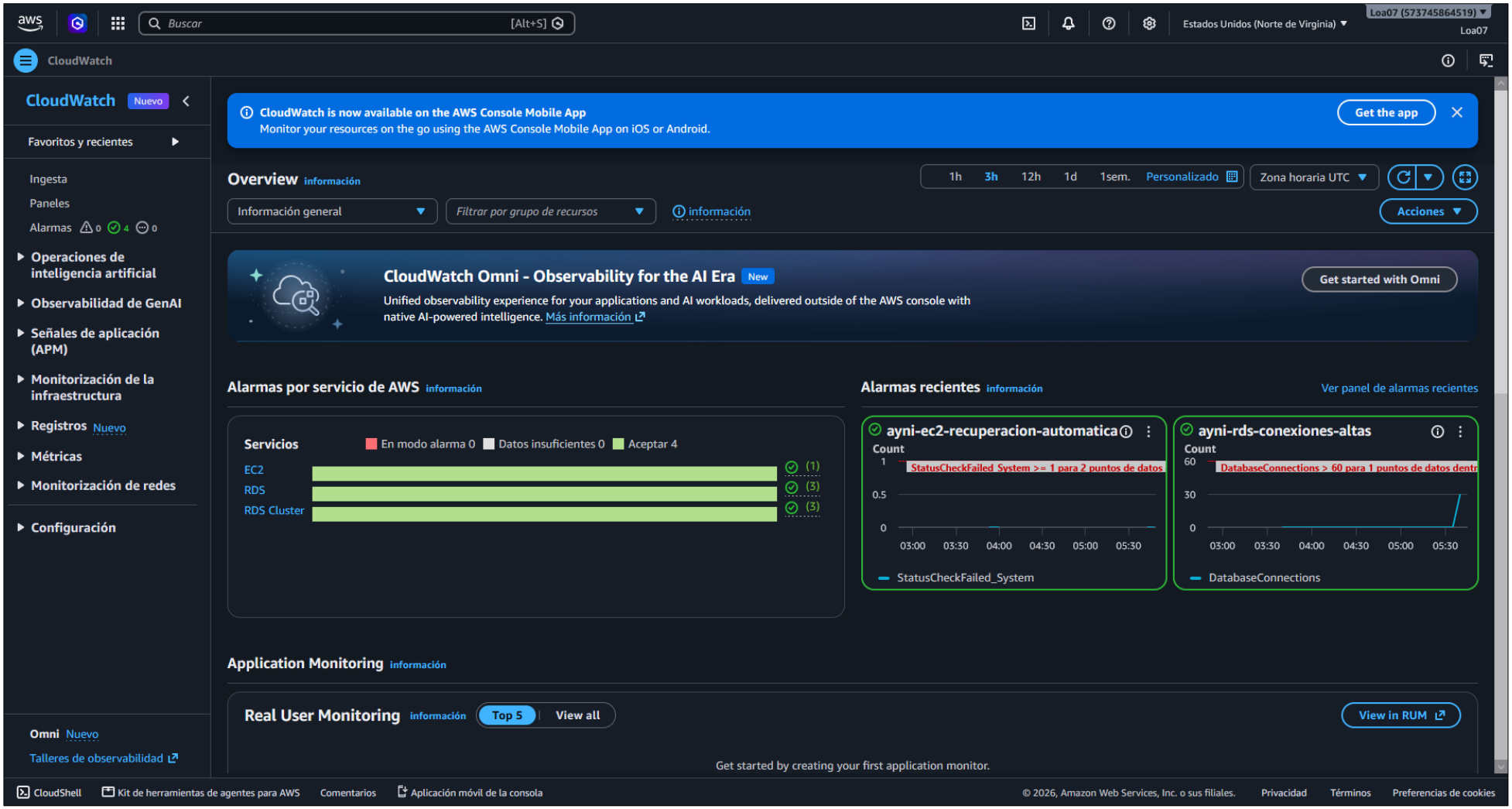



Figura 23 · Alarmas de CloudWatch con aviso por correo mediante SNS.

### 4.4.3 Salud de la aplicación en Grafana

Prometheus recoge cada quince segundos las métricas de los cinco servicios y de la instancia, y Grafana las presenta en un tablero. La fila superior resume lo esencial: servicios arriba, disponibilidad de las últimas veinticuatro horas, peticiones por segundo, tasa de errores 5xx y latencia p95. Debajo se ve el tráfico y la latencia por servicio, la latencia del servicio que hace el OCR del DNI, la memoria de cada servicio, las conexiones a RDS y el consumo de CPU, memoria, disco y red de la instancia.




Figura 24 · Grafana en producción: salud técnica de las últimas 6 horas.

La disponibilidad de las últimas veinticuatro horas no llega al cien por ciento porque cada despliegue reinicia los contenedores durante unos segundos; es el coste de publicar varias versiones en un día, y se resolverá con despliegues sin corte cuando haya más de una instancia.

### 4.4.4 Tareas de administración

Administrar la base no es solo mirar métricas: también hay que poder recuperarla, cambiar su estructura sin perder datos y controlar quién entra. Ninguna de estas tareas se hace a mano sobre la base: todas pasan por RDS, por migraciones versionadas o por los secretos de AWS. La tabla resume cómo se hace cada una:

**Tabla 23** · Tareas de administración de la base de datos

| Tarea de administración | Cómo se hace |
|---|---|
| Respaldo | Automático diario + PITR de RDS; snapshot final al eliminar la instancia |
| Restauración | Consola RDS → Restore to point in time (crea una instancia nueva) |
| Cambios de esquema | Solo por migraciones Flyway versionadas y revisadas en PR |
| Accesos | Usuario ayni_app sin superusuario; la cuenta maestra en Secrets Manager |
| Alertas | CPU, almacenamiento y conexiones en CloudWatch → SNS al correo del equipo |
| Correo en sandbox | SES solo entrega a correos verificados mientras está en sandbox. En producción real se pedirá el acceso de producción y un dominio propio con SPF y DKIM. |

*Nota.* Elaboración propia.

### 4.4.5 Los datos de producción en la base

Las métricas dicen que la base está sana, pero no que la aplicación escribe en ella. Para comprobarlo se consultó la base de producción desde la propia instancia (Session Manager, sin exponer la base a internet) después de abrir la cuenta y hacer el primer depósito:

- **Cuántos registros hay** en las tablas principales: usuarios, cuentas y asientos.
- **Los asientos de la cuenta abierta,** con su tipo e importe.
- **La partida doble cuadra:** la suma de los cargos es igual a la suma de los abonos.

```
root@ip-172-31-23-91:~# cd /opt/ayni && set -a && . ./.env && set +a
root@ip-172-31-23-91:/opt/ayni# export PGPASSWORD="$DB_CONTRASENA"; DB="host=$DB_HOST dbname=ayni user=$DB_USUARIO sslmode=require"
  psql "$DB" -c "SELECT 'usuarios' AS tabla, count(*) FROM identity.usuario UNION ALL SELECT 'cuentas', count(*) FROM core.cuenta UNION ALL SELECT 'asientos', count(*)
  FROM core.asiento;"
  psql "$DB" -c "SELECT c.numero AS cuenta, a.tipo, a.importe FROM core.asiento a JOIN core.cuenta c ON c.id = a.cuenta_id;"
  psql "$DB" -c "SELECT a.tipo, sum(a.importe) AS total FROM core.asiento a GROUP BY a.tipo;"
  tabla   | count
----------+-------
 usuarios |     1
 cuentas  |     3
 asientos |     2
(3 rows)

     cuenta     | tipo  | importe
----------------+-------+---------
 00110000000001 | CARGO |  300.00
 00111000000001 | ABONO |  300.00
(2 rows)

 tipo  | total
-------+--------
 CARGO | 300.00
 ABONO | 300.00
(2 rows)

root@ip-172-31-23-91:/opt/ayni#
```

Figura 24c · Consulta a la base de producción: registros por tabla, asientos de la cuenta y cuadre de la partida doble.

La consulta muestra un usuario y tres cuentas: la del cliente y las dos cuentas técnicas del banco, la de fondeo y la de compensación interbancaria. El depósito de S/ 300 generó dos asientos: un **cargo** en la cuenta técnica de fondeo y un **abono** en la cuenta del cliente. La suma de cargos (S/ 300) es igual a la de abonos (S/ 300): la partida doble cuadra en producción.

Cada operación que se hace desde la web aparece en la base en el mismo instante, y el saldo que muestra el panel es exactamente la suma de esos asientos.

### Fuentes consultadas


- Amazon Web Services. *AWS Well-Architected Framework* (pilares de fiabilidad y seguridad).
- Humble, J. y Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
- GitHub. *Security hardening with OpenID Connect* (documentación de GitHub Actions).

# 5. VALIDACIÓN DEL SISTEMA

La validación responde a una pregunta concreta: ¿puede un cliente abrir su cuenta y mover su dinero? Se hizo en dos partes. En **producción** se recorrió el camino real de un cliente con un DNI real. Las **transferencias** necesitan dos cuentas abiertas, así que su recorrido completo se validó en el **entorno local**, que ejecuta las mismas imágenes de los servicios con PostgreSQL y RabbitMQ locales.

## 5.1 En producción: una cuenta abierta por el recorrido real

Joaquín hizo su onboarding en producción con su DNI real: captura de los dos lados, lectura del MRZ, confirmación de los datos y selfie. Un operador revisó la selfie contra la foto del DNI y aprobó la solicitud; core-banking recibió el evento de aprobación y abrió la cuenta sola, con su número y su CCI. El primer depósito quedó registrado por partida doble contra la cuenta técnica de fondeo.

Por el camino aparecieron los fallos que se corrigieron en las releases 1.1.2 a 1.1.6 (sección 4.3.2): el DNI azul lleva el MRZ en el anverso, la lectura tardaba más que el tiempo de espera y, tras la selfie, ningún servicio aprobaba la solicitud. Encontrarlos fue precisamente el valor de probar con un documento real.

La tabla cruza el plan de pruebas (sección 4.1) con lo que se validó directamente en la nube:

**Tabla 24** · Validación del plan de pruebas en producción (AWS)

| Aspecto del plan | Validación en producción (AWS) | Resultado |
|---|---|---|
| Registro del cliente | Alta con correo, contraseña y datos declarados | OK |
| Verificación del DNI | Captura de los dos lados, lectura del MRZ y confirmación de datos | OK (tras las releases 1.1.3 a 1.1.5) |
| Selfie y aprobación | Entrega de la selfie y aprobación por un operador | OK |
| Apertura de cuenta | core-banking abre la cuenta al recibir el evento | OK |
| Ingreso con segundo factor | Contraseña + código TOTP de la app autenticadora | OK |
| Depósito y saldo | Depósito registrado por partida doble; saldo calculado desde los asientos | OK |
| Integración con la base | Los registros aparecen en RDS en el momento (figura 24c) | OK |
| Seguridad | HTTPS, cabeceras de seguridad, 401 sin token (pruebas de humo) y escaneo con Kali | OK |
| Transferencia entre clientes | Necesita una segunda cuenta abierta en producción | Validada en local (5.2) |

*Nota.* Elaboración propia.

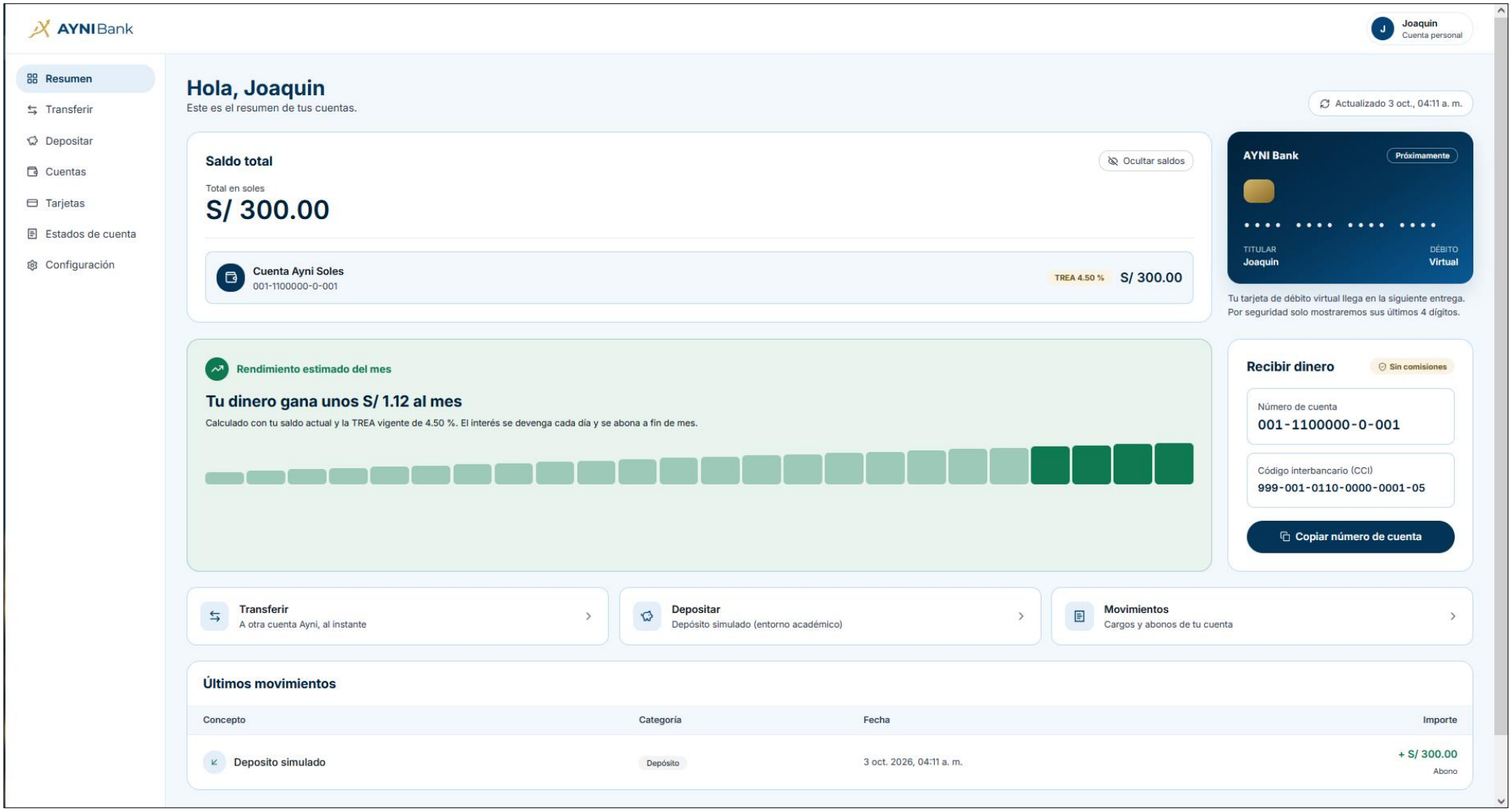



Figura 24a · Producción: la cuenta abierta, con número, CCI y su primer depósito.

La release 1.1.6, la que se grabó en video, publicó la pestaña de **Configuración**. Muestra el perfil del titular con el correo enmascarado, el segundo factor activo y los ajustes de los siguientes sprints marcados como «próximamente»: ningún botón finge hacer algo que todavía no hace.

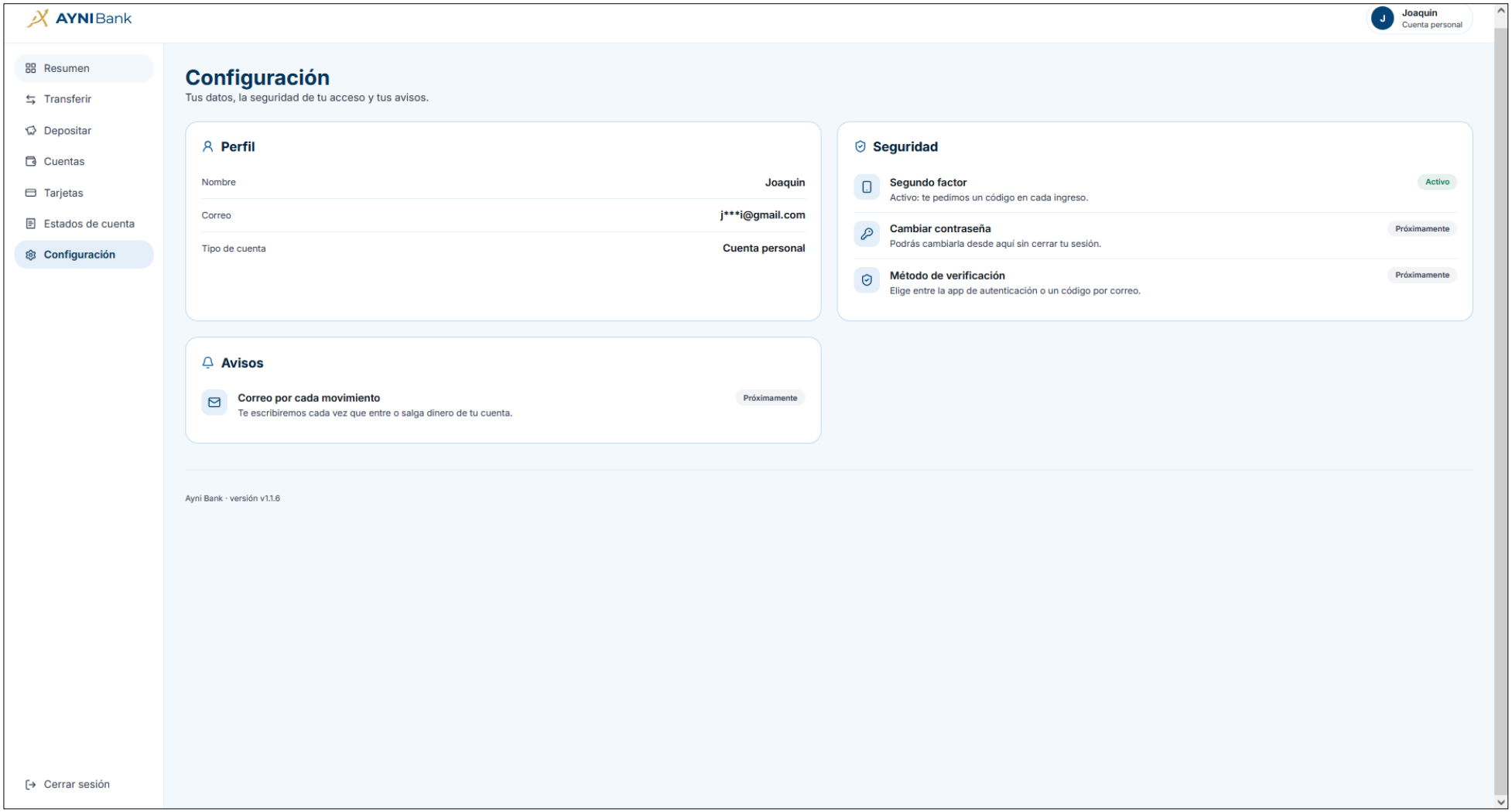



Figura 24b · Producción: la pestaña de Configuración publicada con la release 1.1.6.

## 5.2 En el entorno local: el recorrido completo de las operaciones

Este recorrido usa dos cuentas abiertas y prueba todas las operaciones de dinero, incluido un rechazo. La tabla resume cada paso y su resultado; después se muestra cada uno con su captura.

**Tabla 25** · Recorrido de las operaciones en el entorno local

| Paso del recorrido (local) | Resultado |
|---|---|
| Ingreso con contraseña y código TOTP | Acceso al panel |
| Depósito simulado de S/ 1 500.00 | Abono y comprobante |
| Transferencia de S/ 250.00 a otra cuenta Ayni | Cargo, abono en la otra cuenta y comprobante |
| Transferencia de S/ 90 000.00 | Rechazada por límite, con su motivo |
| Panel final | Saldo S/ 2 500.00 tras dos depósitos de S/ 1 500 y dos transferencias de S/ 250, calculado desde los asientos |

*Nota.* Elaboración propia. Las capturas de cada paso se presentan en las figuras 25 a 30.

### 5.2.1 Punto de partida

El titular entra con su contraseña y el código de su app autenticadora y llega al panel. El saldo se calcula sumando los asientos de la cuenta; no es un número que alguien pueda editar.

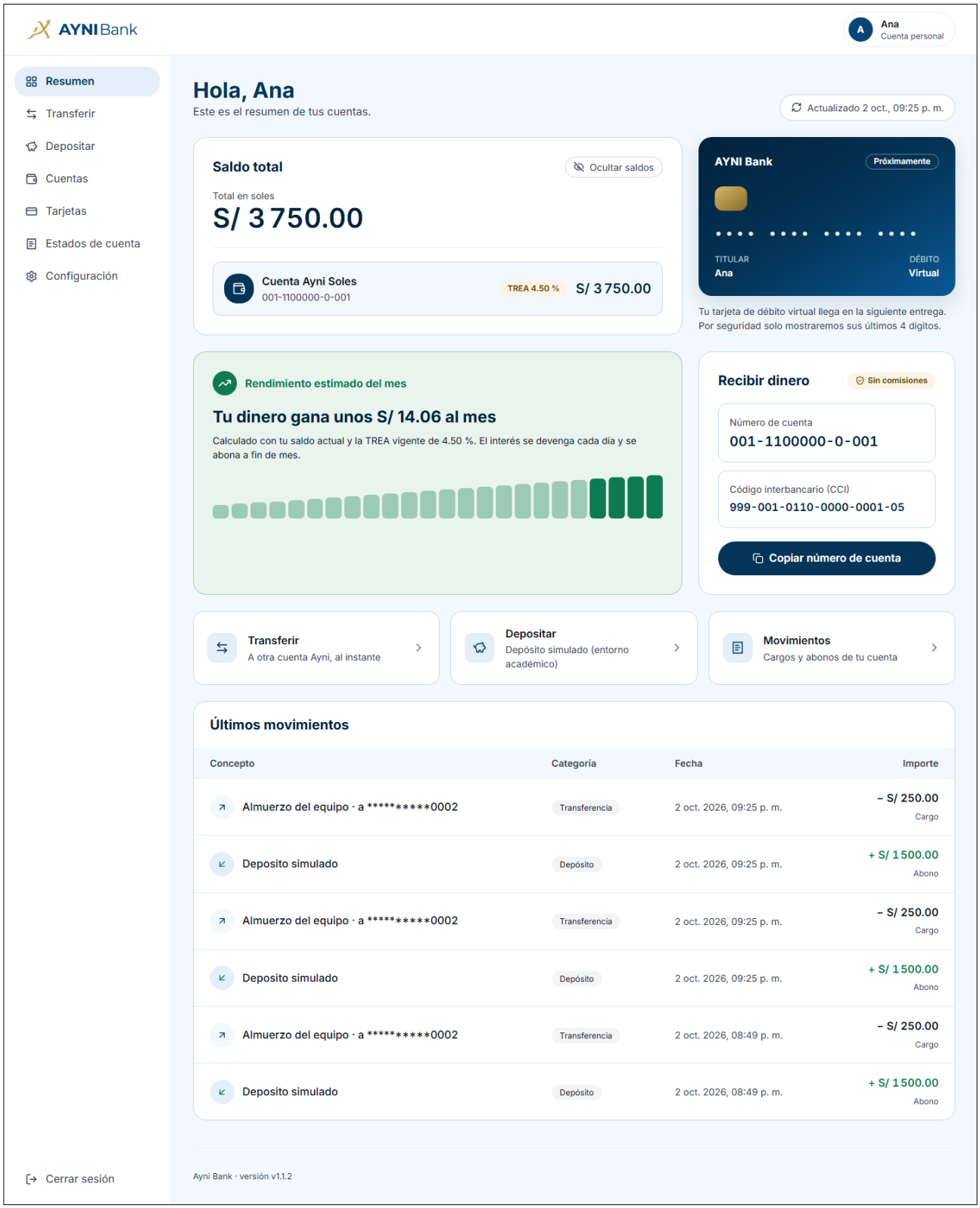



Figura 25 · Panel antes de operar.

### 5.2.2 Depósito

El depósito simulado mueve dinero desde la cuenta técnica de fondeo del banco: un cargo en esa cuenta y un abono en la del cliente, en la misma transacción. El comprobante muestra el importe, la fecha y el número de operación.

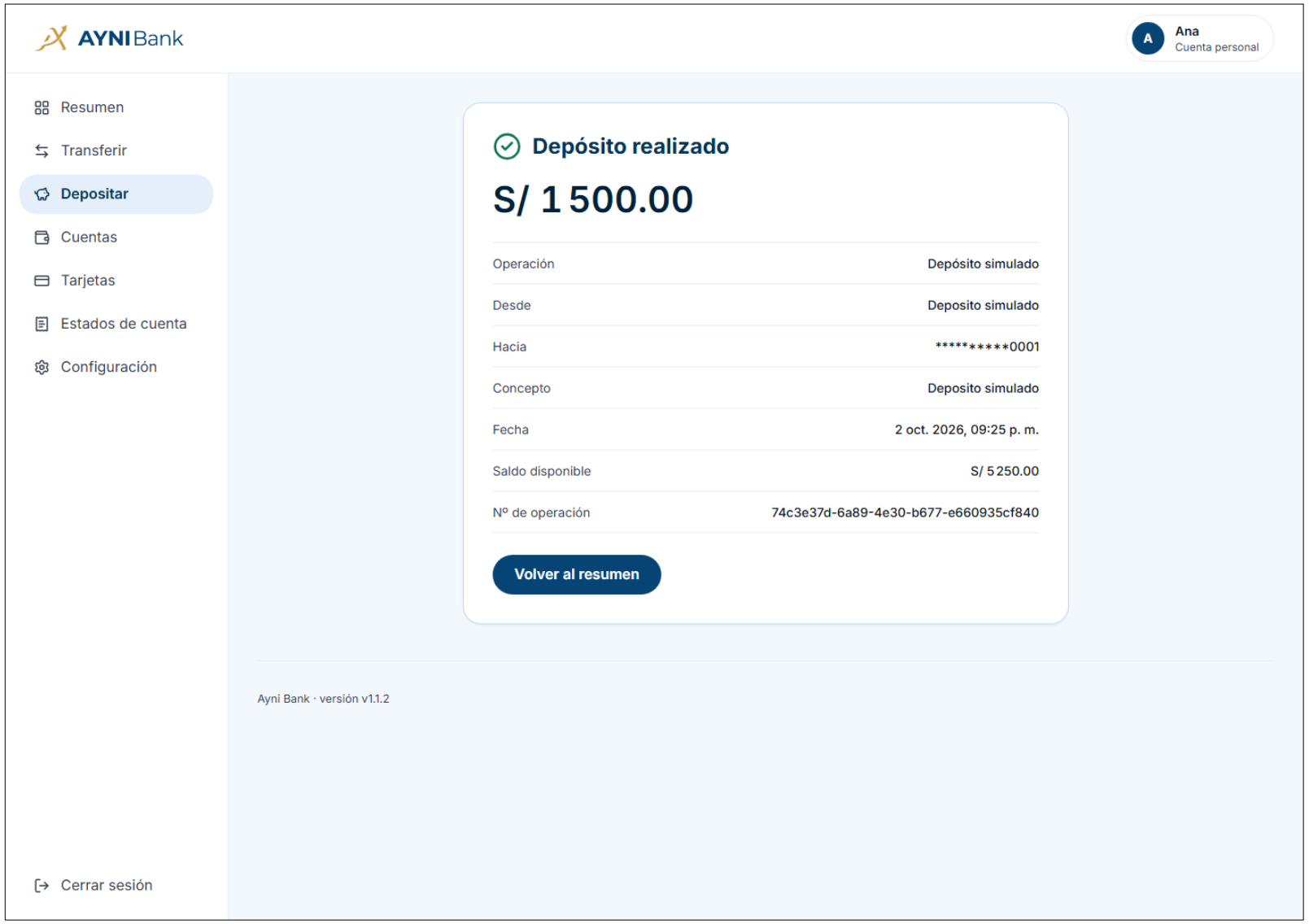



Figura 26 · Comprobante del depósito.

### 5.2.3 Transferencia entre cuentas Ayni

Antes de mover el dinero, la pantalla muestra a quién se envía y cuánto, para que el cliente lo confirme. Cada intento lleva una clave de idempotencia: si la conexión falla y el cliente vuelve a confirmar, la transferencia no se repite.

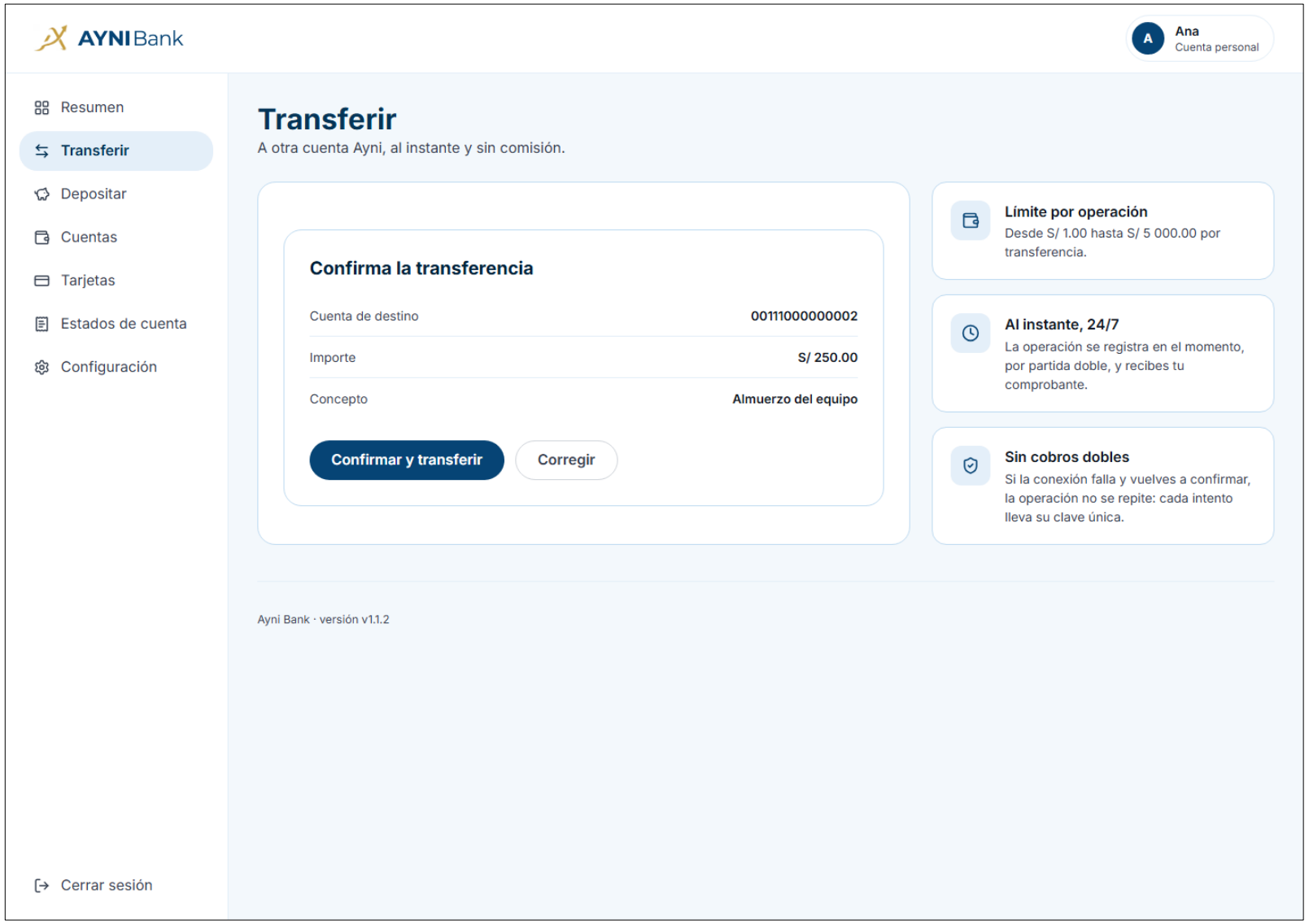



Figura 27 · Confirmación de la transferencia.

Al confirmar, se registran juntos el cargo en la cuenta de origen y el abono en la de destino. Si cualquiera de los dos fallara, no se guardaría ninguno.

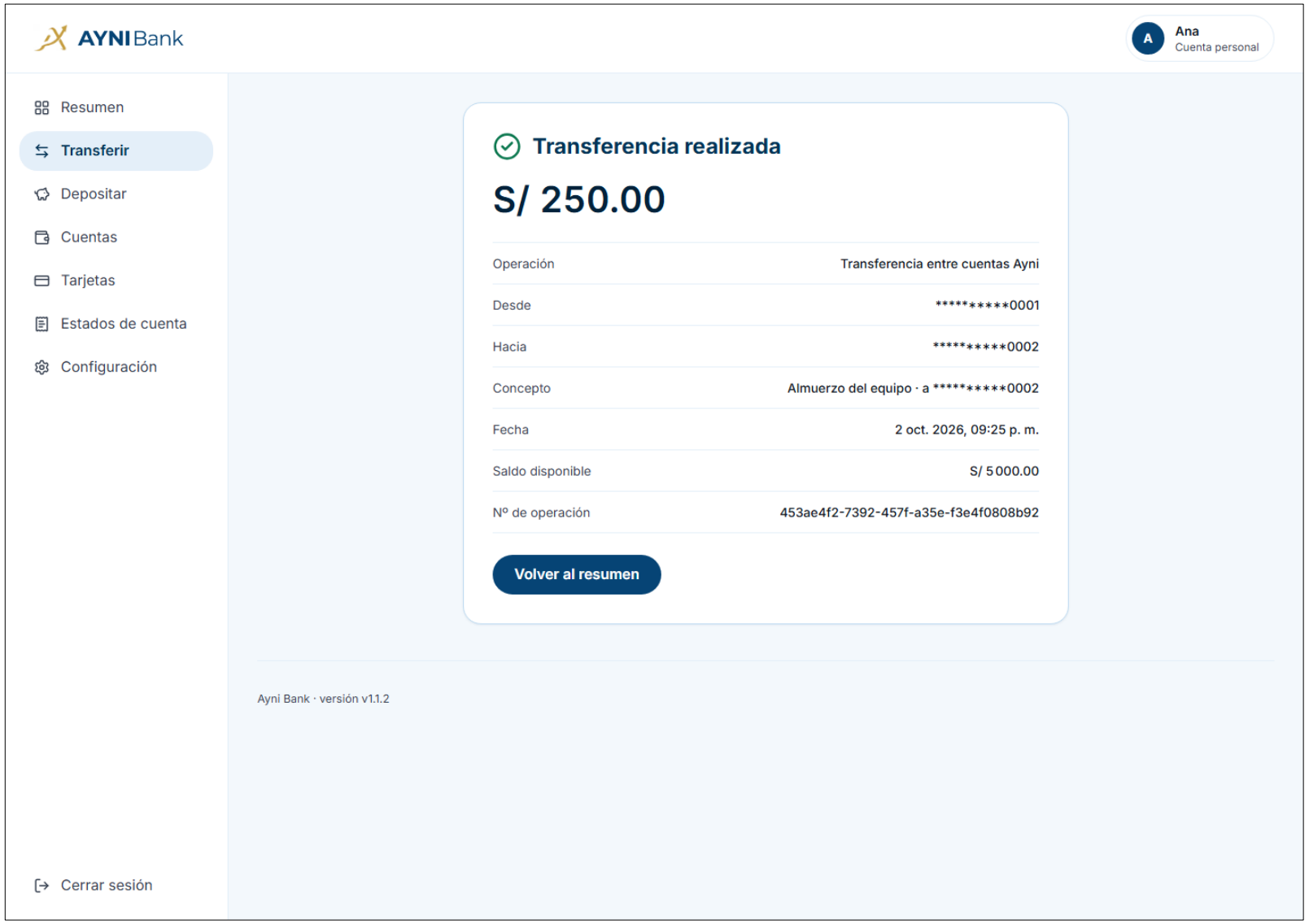



Figura 28 · Comprobante de la transferencia.

### 5.2.4 Rechazo por límite

Una transferencia por encima del límite por operación se rechaza con un motivo comprensible para el cliente. El rechazo no deja asientos, pero sí queda registrado en la tabla de auditoría de core, como se explicó en la sección 2.2.4.

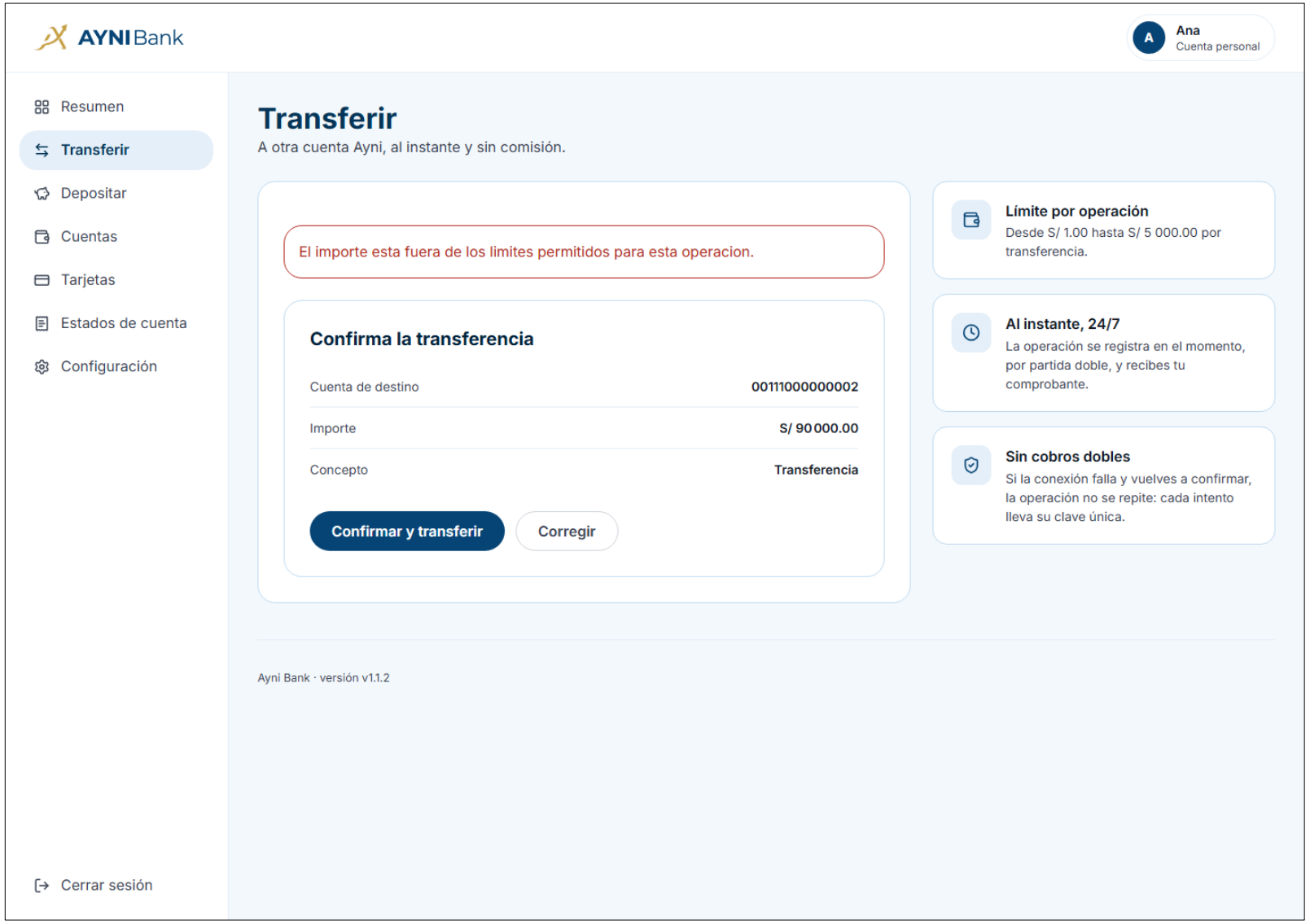



Figura 29 · Transferencia rechazada por límite.

### 5.2.5 Resultado final

Al terminar, el panel muestra todos los movimientos y un saldo que coincide exactamente con la suma de los asientos: dos depósitos de S/ 1 500 y dos transferencias de S/ 250 dejan S/ 2 500.

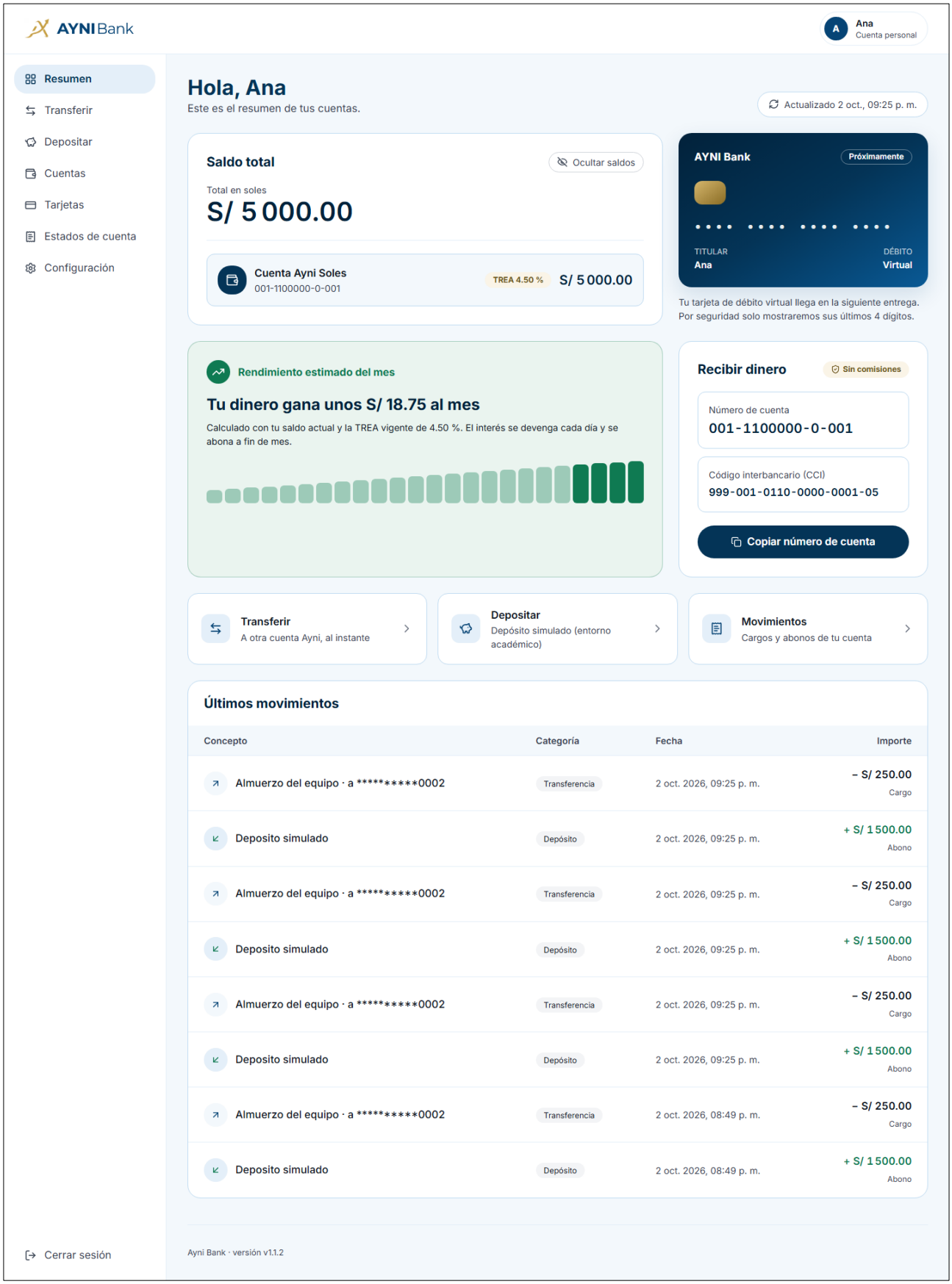



Figura 30 · Panel final con los movimientos y la versión desplegada.

# 6. TECNOLOGÍAS UTILIZADAS EN CADA PUNTO

Este capítulo resume, punto por punto de la rúbrica, qué tecnología se usó. Las elegimos con tres criterios:

- **Estándares de la industria bancaria:** PostgreSQL, Spring Boot y AWS son habituales en entidades financieras.
- **Gratuitas o dentro del plan gratuito:** el proyecto no tiene presupuesto, y el costo de AWS se mantuvo en cero.
- **Verificables:** cada tecnología deja evidencia (un informe, una métrica o una prueba) que se puede revisar.

**Tabla 26** · Tecnologías utilizadas en cada punto de la rúbrica

| Punto de la rúbrica | Tecnologías |
|---|---|
| Base de datos | PostgreSQL 17 en Amazon RDS · Flyway · Spring Data JPA / Hibernate · KMS |
| Patrón de acceso | Repository sobre arquitectura hexagonal · Spring Data JPA · ArchUnit |
| Autenticación | Spring Boot 3 / Java 21 · Argon2id · TOTP (RFC 6238) · JWT HS256 (Nimbus) · Spring Cloud Gateway · Amazon SES |
| Cifrado | TLS 1.3 (Let's Encrypt, Caddy) · AES-256-GCM · SHA-256 · KMS · Parameter Store · Secrets Manager |
| Pruebas de seguridad | OWASP ZAP · Nmap · Nikto · testssl.sh (Kali Linux) en GitHub Actions · Trivy · gitleaks · SonarCloud |
| Pruebas del sistema | JUnit 5 · Mockito · AssertJ · Cucumber · ArchUnit · JaCoCo · pytest · Vitest · Playwright · axe-core |
| Despliegue | GitHub Actions · Docker · Amazon ECR · EC2 · Systems Manager · CloudFormation · OIDC |
| Monitoreo | Prometheus · Grafana · node-exporter · Micrometer · CloudWatch · SNS · AWS Budgets |
| Aplicación | Next.js / React / TypeScript · FastAPI / Python (OCR con OpenCV y PaddleOCR) · RabbitMQ |
| Gestión | Jira (Scrum) · GitHub (GitFlow, PR revisados) · ADR en el repositorio |

*Nota.* Elaboración propia.

# 7. CONCLUSIONES

- El sistema está **desplegado y funcionando en AWS**, con despliegue continuo, pruebas de humo y rollback automático que se probó en una situación real.
- La base de datos es coherente con el negocio: el modelo físico, el DDL y el diagrama salen de la misma fuente, y la partida doble se cumple incluso con otros bancos.
- La seguridad cubre confidencialidad (cifrado en tránsito y en reposo), integridad (partida doble, idempotencia, auditoría) y disponibilidad (monitoreo, alarmas y respaldo), y se verificó con herramientas de Kali contra producción.
- Se levantaron el 100 % de las observaciones del APF1, incluida una respuesta nuestra que era incorrecta.
- Usar el sistema en producción con un DNI real destapó fallos que las pruebas no veían; se corrigieron en cinco releases (1.1.2 a 1.1.6) por el mismo pipeline, sin tocar el servidor a mano, y la última se grabó en video.
- Lo que queda abierto se declara: cotejo facial automático de la selfie (hoy lo aprueba un operador), segundo factor dentro del registro (HU-22), replicación Multi-AZ, dominio propio con SES en producción, filtro de permisos por endpoint y retirar 'unsafe-inline' de la CSP.